

```

import React, { useState, useEffect, useMemo, createContext, useContext } from
"react";
import { createRoot } from "react-dom/client";
import {
  Home,
  CalendarDays,
  Heart,
  Wallet,
  TrendingUp,
  Plus,
  Check,
  Trash2,
  Pencil,
  Clock,
  Repeat,
  Bell,
  User,
  Settings,
  Smile,
  Moon,
  Sparkles,
  BookOpen,
  Target,
  Gift,
  FolderKanban,
  ArrowUpCircle,
  ArrowDownCircle,
  PiggyBank,
  ChevronRight,
  ChevronLeft,
  X,
  Flame,
  Star,
  ListChecks,
  Sun,
  Sunset,
  MoonStar
} from "lucide-react";
import {
  ResponsiveContainer,
  LineChart,
  Line,
  BarChart,
  Bar,
  XAxis,
  YAxis,
  CartesianGrid,
  Tooltip
} from "recharts";
const C = {
  bg: "#FFF6F8",
  card: "#FFFFFF",
  rosaClaro: "#FDE6EC",
  rosaChá: "#F3C6D3",
  malva: "#E3B8D6",
  lilas: "#CBB4E0",
  champagne: "#CBA76B",
  texto: "#5B4B57",
  textoSuave: "#9A8A95",
  verde: "#9CC7A1",
  vermelho: "#E3A0A0"
};
const estiloInput = {
  width: "100%",
  padding: "10px 14px",
  borderRadius: 14,

```

```

border: `1px solid ${C.rosaClaro}` ,
fontSize: 14,
color: C.texto,
outline: "none",
background: "#FFBFC",
boxSizing: "border-box"
};
const CHAVE_ARMAZENAMENTO = "meuPlannerDados_v1";
function carregarEstadoSalvo() {
  try {
    const bruto = window.localStorage.getItem(CHAVE_ARMAZENAMENTO);
    if (!bruto) return null;
    const dados = JSON.parse(bruto);
    if (!dados || typeof dados !== "object") return null;
    return dados;
  } catch (erro) {
    console.warn("N\uxE3o foi poss\uxEDvel carregar os dados salvos no dispositivo:",
erro);
    return null;
  }
}
function salvarEstado(estado) {
  try {
    window.localStorage.setItem(CHAVE_ARMAZENAMENTO, JSON.stringify(estado));
  } catch (erro) {
    console.warn("N\uxE3o foi poss\uxEDvel salvar os dados no dispositivo:", erro);
  }
}
let contador = 0;
const uid = () => `id-${Date.now()}-${contador++}`;
function isoDeHoje(offset = 0) {
  const d = /* #__PURE__ */ new Date();
  d.setDate(d.getDate() + offset);
  return d.toISOString().slice(0, 10);
}
const HOJE_ISO = isoDeHoje(0);
function formatarDataLonga(iso) {
  const d = /* #__PURE__ */ new Date(iso + "T12:00:00");
  const s = d.toLocaleDateString("pt-BR", { weekday: "long", day: "numeric", month:
"long" });
  return s.charAt(0).toUpperCase() + s.slice(1);
}
function diaCurto(iso) {
  const d = /* #__PURE__ */ new Date(iso + "T12:00:00");
  return d.toLocaleDateString("pt-BR", { weekday: "short" }).replace(".", "");
}
function numeroDia(iso) {
  return (/* #__PURE__ */ new Date(iso + "T12:00:00")).getDate();
}
function indiceSemana(iso) {
  const d = (/* #__PURE__ */ new Date(iso + "T12:00:00")).getDay();
  return (d + 6) % 7;
}
function formatBRL(v) {
  return (v || 0).toLocaleString("pt-BR", { style: "currency", currency: "BRL" });
}
function diasEntre(isoAlvo) {
  const hoje = /* #__PURE__ */ new Date(HOJE_ISO + "T00:00:00");
  const alvo = /* #__PURE__ */ new Date(isoAlvo + "T00:00:00");
  return Math.round((alvo - hoje) / 864e5);
}
const DIAS_SEMANA = ["Seg", "Ter", "Qua", "Qui", "Sex", "S\uxE1b", "Dom"];
const FRASES = [
  "Cuidar de voc\uxEA tamb\uxE9m \xE9 produtividade.",
  "Pequenos passos, todos os dias, mudam tudo.",
  "Organizar sua rotina \xE9 um ato de amor-pr\uxF3prio.",

```

```

    "Voc\xEA n\xE3o precisa ser perfeita, s\xF3 constante.",
    "Hoje \xE9 um bom dia para recomen\xE7ar com leveza.",
    "Progresso \xE9 progresso, mesmo devagar.",
    "Respire. Voc\xEA est\xE1 indo bem."
  ];
  const AVATARES = ["\u{1F338}", "\u{1F337}", "\u{1F98B}", "\u2728", "\u{1F319}",
    "\u{1F351}", "\u{1FAB7}"];
  const CATEGORIAS_TAREFA = ["Trabalho", "Pessoal", "Casa", "Sa\xFAde", "Estudos",
    "Outros"];
  const PRIORIDADES = ["Alta", "M\xE9dia", "Baixa"];
  const CORES_PRIORIDADE = { Alta: C.vermelho, M\u00E9dia: C.champagne, Baixa: C.verde
  };
  const CATEGORIAS_META = ["Pessoal", "Carreira", "Sa\xFAde", "Financeiro", "Outros"];
  const CATEGORIAS_RECEITA = ["Sal\xE1rio", "Freelance", "Presente", "Outros"];
  const CATEGORIAS_DESPESA = ["Alimenta\xE7\xE3o", "Transporte", "Moradia", "Lazer",
    "Sa\xFAde", "Compras", "Assinaturas", "Outros"];
  const CONTAS = ["Carteira", "Conta Corrente", "Poupan\xE7a", "Cart\xE3o de
    Cr\xE9dito"];
  const DIAS_SEMANA_CAL = ["D", "S", "T", "Q", "Q", "S", "S"];
  const TIPOS_ESCALA = [
    { valor: "plantao", emoji: "\u{1F319}", rotulo: "Plant\xE3o Noturno", cor: C.lilas,
      corFundo: C.lilas + "40" },
    { valor: "nao_trabalho", emoji: "\u{1F6AB}", rotulo: "N\xE3o trabalho", cor:
      C.vermelho, corFundo: C.vermelho + "30" },
    { valor: "folga", emoji: "\u2600\uFE0F", rotulo: "Folga", cor: C.verde, corFundo:
      C.verde + "40" },
    { valor: "ferias", emoji: "\u2728", rotulo: "F\xE9rias", cor: C.champagne, corFundo:
      C.champagne + "40" },
    { valor: "troca", emoji: "\u{1F504}", rotulo: "Troca de plant\xE3o", cor: C.malva,
      corFundo: C.malva + "40" }
  ];
  function tipoEscalaPorValor(v) {
    return TIPOS_ESCALA.find((t) => t.valor === v) || null;
  }
  function diasNoMes(ano, mes) {
    return new Date(ano, mes + 1, 0).getDate();
  }
  function primeiroDiaSemanaMes(ano, mes) {
    return new Date(ano, mes, 1).getDay();
  }
  const HUMORES = [
    { valor: "otima", emoji: "\u{1F60A}", rotulo: "\xD3tima" },
    { valor: "bem", emoji: "\u{1F60C}", rotulo: "Bem" },
    { valor: "neutra", emoji: "\u{1F610}", rotulo: "Neutra" },
    { valor: "cansada", emoji: "\u{1F614}", rotulo: "Cansada" },
    { valor: "dificil", emoji: "\u{1F62B}", rotulo: "Dif\xEDcil" }
  ];
  function tarefaOcorreEm(t, iso) {
    if (!t.recorrente) return t.data === iso;
    if (t.frequencia === "diaria") return iso >= t.data;
    if (t.frequencia === "semanal") return iso >= t.data && (t.diasSemana ||
    []).includes(indiceSemana(iso));
    return t.data === iso;
  }
  function tarefaConcluidaEm(t, iso) {
    if (!t.recorrente) return !t.concluida && t.data === iso;
    return !(t.historico && t.historico[iso]);
  }
  function dadosIniciais() {
    return {
      perfil: null,
      // dispara onboarding se null
      humorHoje: null,
      tarefas: [
        { id: uid(), titulo: "Reuni\xE3o com equipe", categoria: "Trabalho", prioridade:
        "Alta", hora: "09:00", data: HOJE_ISO, recorrente: false, concluida: false, lembrete:

```

```

true, historico: {} },
  { id: uid(), titulo: "Beber 2L de \xE1gua", categoria: "Sa\xFAde", prioridade:
"M\xE9dia", hora: "08:00", data: HOJE_ISO, recorrente: true, frequencia: "diaria",
diasSemana: [], concluida: false, lembrete: false, historico: {} },
  { id: uid(), titulo: "Pagar conta de luz", categoria: "Casa", prioridade:
"Alta", hora: "18:00", data: HOJE_ISO, recorrente: false, concluida: false, lembrete:
true, historico: {} },
  { id: uid(), titulo: "Ler 10 p\xE1ginas", categoria: "Estudos", prioridade:
"Baixa", hora: "20:00", data: HOJE_ISO, recorrente: false, concluida: true, lembrete:
false, historico: {} }
],
  habitos: [
    { id: uid(), nome: "Beber \xE1gua", frequencia: "Di\xE1rio", lembrete: true,
historico: { [isoDeHoje(-2)]: true, [isoDeHoje(-1)]: true } },
    { id: uid(), nome: "Alongar-se", frequencia: "Di\xE1rio", lembrete: false,
historico: { [isoDeHoje(-1)]: true } },
    { id: uid(), nome: "Ler antes de dormir", frequencia: "Semanal", lembrete:
false, historico: {} }
],
  autocuidado: [
    { id: uid(), nome: "Skincare da noite", feitoHoje: false },
    { id: uid(), nome: "10 min de respira\xE7\xE3o", feitoHoje: false },
    { id: uid(), nome: "Momento sem telas", feitoHoje: false }
],
  diario: [
    { id: uid(), data: isoDeHoje(-1), texto: "Dia corrido, mas consegui organizar
minha semana.", humor: "bem" }
],
  desejos: [
    { id: uid(), titulo: "Fim de semana na praia", categoria: "Viagem", preco: 800 }
],
  projetos: [
    { id: uid(), titulo: "Projeto Bem-Estar", etapasTotal: 15, etapasConcluidas: 4 }
],
  metas: [
    { id: uid(), titulo: "Organizar o guarda-roupa", categoria: "Pessoal", prazo:
isoDeHoje(10), progresso: 40 },
    { id: uid(), titulo: "Concluir curso online", categoria: "Carreira", prazo:
isoDeHoje(20), progresso: 65 }
],
  transacoes: [
    { id: uid(), tipo: "Receita", descricao: "Sal\xE1rio", categoria: "Sal\xE1rio",
conta: "Conta Corrente", valor: 3200, data: isoDeHoje(-5) },
    { id: uid(), tipo: "Despesa", descricao: "Supermercado", categoria:
"Alimenta\xE7\xE3o", conta: "Cart\xE3o de Cr\xE9dito", valor: 420, data: isoDeHoje(-3)
},
    { id: uid(), tipo: "Despesa", descricao: "Uber", categoria: "Transporte", conta:
"Carteira", valor: 38, data: isoDeHoje(-1) },
    { id: uid(), tipo: "Despesa", descricao: "Assinatura streaming", categoria:
"Assinaturas", conta: "Cart\xE3o de Cr\xE9dito", valor: 45, data: isoDeHoje(-2) }
],
  diasEscala: {},
  horasPorPlantao: 12,
  missoesFeitas: {},
  pontosHoje: 0,
  pontosTotais: 0,
  rodadasCompleatas: 0,
  conquistas: [],
  coposAgua: 0,
  porcoesFrutas: 0,
  pesoAtual: 70,
  dieta: { cafeDaManha: "", almoco: "", jantar: "" },
  orcamentoMensal: 2200,
  metasFinanceiras: [
    { id: uid(), titulo: "Reserva de emerg\xEAncia", valorAlvo: 5e3, valorAtual:
1800, prazo: isoDeHoje(120) }
]

```

```

    ],
    historicoDemo: [
      { dia: "5 dias atr\xE1s", pct: 62 },
      { dia: "4 dias atr\xE1s", pct: 70 },
      { dia: "3 dias atr\xE1s", pct: 55 },
      { dia: "2 dias atr\xE1s", pct: 80 },
      { dia: "Ontem", pct: 75 }
    ]
  };
}
const AppCtx = createContext(null);
const useApp = () => useContext(AppCtx);
function Cartao({ children, style, onClick }) {
  return /* #__PURE__ */ React.createElement(
    "div",
    {
      onClick,
      style: { background: C.card, borderRadius: 20, boxShadow: "0 2px 12px rgba(203,180,224,0.14)", ...style }
    },
    children
  );
}
function Pill({ children, cor, ativo = true }) {
  return /* #__PURE__ */ React.createElement(
    "span",
    {
      style: {
        fontSize: 11,
        fontWeight: 600,
        padding: "4px 10px",
        borderRadius: 999,
        background: ativo ? cor + "22" : C.rosaClaro,
        color: ativo ? cor : C.textoSuave
      }
    },
    children
  );
}
function BotaoFlutuante({ onClick }) {
  return /* #__PURE__ */ React.createElement(
    "button",
    {
      onClick,
      style: {
        position: "fixed",
        bottom: 84,
        right: "50%",
        transform: "translateX(160px)",
        width: 52,
        height: 52,
        borderRadius: 26,
        border: "none",
        background: `linear-gradient(135deg, ${C.rosaChá}, ${C.lilas})`,
        boxShadow: "0 6px 16px rgba(203,180,224,0.5)",
        display: "flex",
        alignItems: "center",
        justifyContent: "center",
        zIndex: 40
      }
    },
    /* #__PURE__ */ React.createElement(Plus, { size: 24, color: "#fff" })
  );
}
function ProgressoBarra({ pct, cor = C.champagne, alto = 8 }) {
  return /* #__PURE__ */ React.createElement("div", { style: { width: "100%", height:

```

```

alto, borderRadius: 999, background: C.rosaClaro } }, /* @__PURE__ */
React.createElement("div", { style: { width: `${Math.min(100, Math.max(0, pct))}%`,
height: "100%", borderRadius: 999, background: cor, transition: "width .3s" } }));
}
function ProgressoCirculo({ pct, size = 60, stroke = 7, cor = C.champagne }) {
  const r = (size - stroke) / 2, c = 2 * Math.PI * r;
  return /* @__PURE__ */ React.createElement("svg", { width: size, height: size }, /*
@__PURE__ */ React.createElement("circle", { cx: size / 2, cy: size / 2, r, stroke:
C.rosaClaro, strokeWidth: stroke, fill: "none" })), /* @__PURE__ */
React.createElement(
  "circle",
  {
    cx: size / 2,
    cy: size / 2,
    r,
    stroke: cor,
    strokeWidth: stroke,
    fill: "none",
    strokeDasharray: c,
    strokeDashoffset: c - Math.min(100, pct) / 100 * c,
    strokeLinecap: "round",
    transform: `rotate(-90 ${size / 2} ${size / 2})`,
    style: { transition: "stroke-dashoffset .4s" }
  }
), /* @__PURE__ */ React.createElement("text", { x: "50%", y: "53%", textAnchor:
"middle", fontSize: size * 0.24, fontWeight: "600", fill: C.texto }, Math.round(pct),
"%"));
}
function ItemLinha({ icone: Icone, corIcone = C.malva, titulo, subtitulo, direita,
onClick, riscado }) {
  return /* @__PURE__ */ React.createElement(Cartao, { style: { padding: 14, display:
"flex", alignItems: "center", gap: 12 }, onClick }, Icone && /* @__PURE__ */
React.createElement("div", { style: { width: 38, height: 38, borderRadius: 19,
background: corIcone + "22", display: "flex", alignItems: "center", justifyContent:
"center", flexShrink: 0 } }, /* @__PURE__ */ React.createElement(Icone, { size: 17,
color: corIcone })), /* @__PURE__ */ React.createElement("div", { style: { flex: 1,
minWidth: 0 } }, /* @__PURE__ */ React.createElement("p", { style: { fontSize: 14,
fontWeight: 500, color: C.texto, margin: 0, textDecoration: riscado ? "line-through" :
"none", opacity: riscado ? 0.5 : 1 } }, titulo), subtitulo && /* @__PURE__ */
React.createElement("p", { style: { fontSize: 12, color: C.textoSuaVe, margin: "2px 0
0" } }, subtitulo)), direita);
}
function BotaoIcone({ icone: Icone, onClick, cor = C.textoSuaVe }) {
  return /* @__PURE__ */ React.createElement("button", { onClick, style: { background:
"none", border: "none", padding: 6, cursor: "pointer", display: "flex" } }, /*
@__PURE__ */ React.createElement(Icone, { size: 16, color: cor }));
}
function TituloSecao({ children, acao }) {
  return /* @__PURE__ */ React.createElement("div", { style: { display: "flex",
alignItems: "center", justifyContent: "space-between", margin: "18px 4px 10px" } }, /*
@__PURE__ */ React.createElement("h2", { style: { fontSize: 14, fontWeight: 600,
color: C.texto, margin: 0 } }, children), acao);
}
function EstadoVazio({ texto }) {
  return /* @__PURE__ */ React.createElement("div", { style: { padding: "26px 10px",
textAlign: "center", color: C.textoSuaVe, fontSize: 13 } }, texto);
}
function ModalFormulario({ titulo, campos, valores, onFechar, onSalvar, onExcluir,
corDestaque = C.texto }) {
  const [dados, setDados] = useState(valores);
  const set = (chave, val) => setDados((d) => ({ ...d, [chave]: val }));
  return /* @__PURE__ */ React.createElement("div", { style: { position: "fixed",
inset: 0, zIndex: 50, display: "flex", alignItems: "flex-end", justifyContent:
"center", background: "rgba(91,75,87,0.35)" }, onClick: onFechar }, /* @__PURE__ */
React.createElement("div", { style: { width: "100%", maxWidth: 384, background:
"#fff", borderRadius: "28px 28px 0 0", padding: 22, maxHeight: "85vh", overflowY:

```

```

"auto" }, onClick: (e) => e.stopPropagation() }, /* @__PURE__ */
React.createElement("div", { style: { display: "flex", alignItems: "center",
justifyContent: "space-between", marginBottom: 16 } }, /* @__PURE__ */
React.createElement("h3", { style: { fontSize: 16, fontWeight: 600, color: C.texto,
margin: 0 } }, titulo), /* @__PURE__ */ React.createElement("button", { onClick:
onFechar, style: { background: "none", border: "none" } }, /* @__PURE__ */
React.createElement(X, { size: 20, color: C.textoSuave }))), /* @__PURE__ */
React.createElement("div", { style: { display: "flex", flexDirection: "column", gap:
14 } }, campos.map((c) => /* @__PURE__ */ React.createElement("div", { key: c.chave },
/* @__PURE__ */ React.createElement("label", { style: { fontSize: 12, fontWeight: 500,
color: C.textoSuave, display: "block", marginBottom: 6 } }, c.rotulo), c.tipo ===
"texto" && /* @__PURE__ */ React.createElement("input", { style: estiloInput, value:
dados[c.chave] || "", onChange: (e) => set(c.chave, e.target.value), placeholder:
c.placeholder })), c.tipo === "textarea" && /* @__PURE__ */
React.createElement("textarea", { style: { ...estiloInput, resize: "none" }, rows: 4,
value: dados[c.chave] || "", onChange: (e) => set(c.chave, e.target.value),
placeholder: c.placeholder })), c.tipo === "numero" && /* @__PURE__ */
React.createElement("input", { type: "number", style: estiloInput, value:
dados[c.chave] ?? "", onChange: (e) => set(c.chave, e.target.value === "" ? "" :
Number(e.target.value)), placeholder: c.placeholder })), c.tipo === "data" && /*
@__PURE__ */ React.createElement("input", { type: "date", style: estiloInput, value:
dados[c.chave] || "", onChange: (e) => set(c.chave, e.target.value) })), c.tipo ===
"hora" && /* @__PURE__ */ React.createElement("input", { type: "time", style:
estiloInput, value: dados[c.chave] || "", onChange: (e) => set(c.chave,
e.target.value) })), c.tipo === "select" && /* @__PURE__ */ React.createElement("div",
{ style: { display: "flex", gap: 8, flexWrap: "wrap" } }, c.opcoes.map((op) => /*
@__PURE__ */ React.createElement(
  "button",
  {
    key: op.valor,
    onClick: () => set(c.chave, op.valor),
    style: {
      padding: "7px 13px",
      borderRadius: 999,
      fontSize: 12,
      fontWeight: 500,
      border: "none",
      background: dados[c.chave] === op.valor ? corDestaque : C.rosaClaro,
      color: dados[c.chave] === op.valor ? "#fff" : C.texto
    }
  },
  op.rotulo
))), c.tipo === "multiselect" && /* @__PURE__ */ React.createElement("div", { style:
{ display: "flex", gap: 8, flexWrap: "wrap" } }, c.opcoes.map((op) => {
  const lista = dados[c.chave] || [];
  const ativo = lista.includes(op.valor);
  return /* @__PURE__ */ React.createElement(
    "button",
    {
      key: op.valor,
      onClick: () => set(c.chave, ativo ? lista.filter((v) => v !== op.valor) :
[...lista, op.valor]),
      style: {
        padding: "7px 13px",
        borderRadius: 999,
        fontSize: 12,
        fontWeight: 500,
        border: "none",
        background: ativo ? corDestaque : C.rosaClaro,
        color: ativo ? "#fff" : C.texto
      }
    },
    op.rotulo
  );
})), c.tipo === "toggle" && /* @__PURE__ */ React.createElement(
  "button",

```

```

    {
      onClick: () => set(c.chave, !dados[c.chave]),
      style: {
        padding: "7px 14px",
        borderRadius: 999,
        fontSize: 12,
        fontWeight: 500,
        border: "none",
        background: dados[c.chave] ? corDestaque : C.rosaClaro,
        color: dados[c.chave] ? "#fff" : C.texto
      }
    },
    dados[c.chave] ? "Sim" : "N\uE3o"
  )), /* @__PURE__ */ React.createElement("button", { onClick: () =>
onSalvar(dados), style: { width: "100%", marginTop: 20, padding: "13px 0",
borderRadius: 999, border: "none", fontSize: 14, fontWeight: 600, color: "#fff",
background: corDestaque } }, "Salvar"), onExcluir && /* @__PURE__ */
React.createElement("button", { onClick: onExcluir, style: { width: "100%", marginTop:
10, padding: "11px 0", borderRadius: 999, border: "none", fontSize: 13, fontWeight:
500, color: C.vermelho, background: "transparent" } }, "Excluir"));
}
function Onboarding({ onConcluir }) {
  const [nome, setNome] = useState("");
  const [avatar, setAvatar] = useState(AVATARES[0]);
  const [horario, setHorario] = useState("07:00");
  const [objetivo, setObjetivo] = useState("");
  return /* @__PURE__ */ React.createElement("div", { style: { minHeight: "100vh",
background: `linear-gradient(160deg, ${C.rosaClaro}, ${C.lilas}55)`, display: "flex",
justifyContent: "center", fontFamily: " 'Poppins', 'Segoe UI', sans-serif" } }, /*
@__PURE__ */ React.createElement("div", { style: { width: "100%", maxWidth: 384,
padding: "48px 24px" } }, /* @__PURE__ */ React.createElement("p", { style: {
fontSize: 13, color: C.textoSuaVe, marginBottom: 4 } }, "Bem-vinda ao"), /* @__PURE__
*/ React.createElement("h1", { style: { fontSize: 28, fontWeight: 700, color: C.texto,
margin: "0 0 26px" } }, "Meu Planner"), /* @__PURE__ */ React.createElement(Cartao, {
style: { padding: 20 } }, /* @__PURE__ */ React.createElement("label", { style: {
fontSize: 12, color: C.textoSuaVe, fontWeight: 500 } }, "Como podemos te chamar?"), /*
@__PURE__ */ React.createElement("input", { style: { ..estiloInput, marginTop: 6,
marginBottom: 16 }, value: nome, onChange: (e) => setNome(e.target.value),
placeholder: "Seu nome" } }, /* @__PURE__ */ React.createElement("label", { style: {
fontSize: 12, color: C.textoSuaVe, fontWeight: 500 } }, "Escolha um avatar"), /*
@__PURE__ */ React.createElement("div", { style: { display: "flex", gap: 8, margin:
"8px 0 16px" } }, AVATARES.map((a) => /* @__PURE__ */ React.createElement("button", {
key: a, onClick: () => setAvatar(a), style: { width: 40, height: 40, borderRadius: 20,
fontSize: 18, border: "none", background: avatar === a ? C.champagne : C.rosaClaro }
}, a))), /* @__PURE__ */ React.createElement("label", { style: { fontSize: 12, color:
C.textoSuaVe, fontWeight: 500 } }, "Hor\uE1rio de in\uEDcio do dia"), /* @__PURE__ */
React.createElement("input", { type: "time", style: { ..estiloInput, marginTop: 6,
marginBottom: 16 }, value: horario, onChange: (e) => setHorario(e.target.value) } }, /*
@__PURE__ */ React.createElement("label", { style: { fontSize: 12, color:
C.textoSuaVe, fontWeight: 500 } }, "Seu principal objetivo agora"), /* @__PURE__ */
React.createElement("input", { style: { ..estiloInput, marginTop: 6 }, value:
objetivo, onChange: (e) => setObjetivo(e.target.value), placeholder: "Ex: mais
equil\uEDbrio na rotina" })), /* @__PURE__ */ React.createElement(
  "button",
  {
    disabled: !nome,
    onClick: () => onConcluir({ nome, avatar, horarioInicio: horario, objetivos:
objetivo } ),
    style: {
      width: "100%",
      marginTop: 24,
      padding: "14px 0",
      borderRadius: 999,
      border: "none",
      fontWeight: 600,
      color: "#fff",

```

```

        fontSize: 14,
        background: nome ? `linear-gradient(135deg, ${C.rosaCha}, ${C.lilas})` :
C.rosaClaro,
        opacity: nome ? 1 : 0.7
    }
},
"Come\xE7ar a organizar minha vida"
));
}
function NavegacaoInferior({ ativa, onMudar }) {
    const itens = [
        { id: "inicio", label: "In\xEDcio", icone: Home },
        { id: "dia", label: "Meu Dia", icone: CalendarDays },
        { id: "vida", label: "Vida", icone: Heart },
        { id: "dinheiro", label: "Dinheiro", icone: Wallet },
        { id: "escala", label: "Escala", icone: TrendingUp },
        { id: "desafios", label: "Desafios", icone: Star }
    ];
    return /* __PURE__ */ React.createElement("div", { style: { position: "fixed",
bottom: 0, left: "50%", transform: "translateX(-50%)", width: "100%", maxWidth: 384,
background: "#fff", borderTop: `1px solid ${C.rosaClaro}`, display: "flex", padding:
"8px 4px calc(8px + env(safe-area-inset-bottom))", zIndex: 30 } }, itens.map((it) => {
        const Icone = it.icone;
        const ativo = ativa === it.id;
        return /* __PURE__ */ React.createElement("button", { key: it.id, onClick: () =>
onMudar(it.id), style: { flex: 1, background: "none", border: "none", display: "flex",
flexDirection: "column", alignItems: "center", gap: 3, padding: "4px 0" } }, /*
__PURE__ */ React.createElement(Icone, { size: 19, color: ativo ? C.champagne :
C.textoSuaVe }), /* __PURE__ */ React.createElement("span", { style: { fontSize: 10,
fontWeight: ativo ? 600 : 400, color: ativo ? C.champagne : C.textoSuaVe } },
it.label));
    }));
}
function Cabecalho({ titulo, subtitulo, onAbrirNotificacoes, onAbrirPerfil,
qtdNotificacoes }) {
    const { perfil } = useApp();
    return /* __PURE__ */ React.createElement("div", { style: { padding: "22px 20px
16px", display: "flex", alignItems: "center", justifyContent: "space-between" } }, /*
__PURE__ */ React.createElement("div", null, /* __PURE__ */
React.createElement("h1", { style: { fontSize: 20, fontWeight: 700, color: C.texto,
margin: 0 } }, titulo), subtitulo && /* __PURE__ */ React.createElement("p", { style:
{ fontSize: 12, color: C.textoSuaVe, margin: "2px 0 0" } }, subtitulo)), /* __PURE__
*/ React.createElement("div", { style: { display: "flex", gap: 10, alignItems:
"center" } }, /* __PURE__ */ React.createElement("button", { onClick:
onAbrirNotificacoes, style: { position: "relative", background: C.rosaClaro, border:
"none", width: 36, height: 36, borderRadius: 18, display: "flex", alignItems:
"center", justifyContent: "center" } }, /* __PURE__ */ React.createElement(Bell, {
size: 16, color: C.texto })), qtdNotificacoes > 0 && /* __PURE__ */
React.createElement("span", { style: { position: "absolute", top: -2, right: -2,
width: 15, height: 15, borderRadius: 8, background: C.vermelho, color: "#fff",
fontSize: 9, display: "flex", alignItems: "center", justifyContent: "center" } },
qtdNotificacoes)), /* __PURE__ */ React.createElement("button", { onClick:
onAbrirPerfil, style: { background: `linear-gradient(135deg, ${C.rosaCha},
${C.lilas})`, border: "none", width: 36, height: 36, borderRadius: 18, display:
"flex", alignItems: "center", justifyContent: "center", fontSize: 16 } }, perfil?.avat
ar || "\u{1F338}")));
}
function useSugestoes() {
    const { tarefas, habitos, metas, transacoes, orcamentoMensal } = useApp();
    return useMemo(() => {
        const lista = [];
        const tarefasHoje = tarefas.filter((t) => tarefaOcorreEm(t, HOJE_ISO));
        const pendentes = tarefasHoje.filter((t) => !tarefaConcluidaEm(t, HOJE_ISO));
        if (pendentes.length > 0) lista.push({ icone: ListChecks, texto: `Voc\xEA tem
${pendentes.length} tarefa(s) pendente(s) hoje.` });
        const altaPendente = pendentes.find((t) => t.prioridade === "Alta");
    });
}

```

```

    if (altaPendente) lista.push({ icone: Flame, texto: `Prioridade alta:
"${altaPendente.titulo}" ainda n\u00E3o foi feita.` });
    const habitosNaoFeitos = habitos.filter((h) => !h.historico[HOJE_ISO]);
    if (habitosNaoFeitos.length > 0) lista.push({ icone: Repeat, texto: `H\u00E1bito
pendente hoje: ${habitosNaoFeitos[0].nome}.` });
    const metaProxima = metas.find((m) => m.progresso < 100 && diasEntre(m.prazo) <= 7
&& diasEntre(m.prazo) >= 0);
    if (metaProxima) lista.push({ icone: Target, texto: `Meta "${metaProxima.titulo}"
vence em ${diasEntre(metaProxima.prazo)} dia(s).` });
    const mesAtual = HOJE_ISO.slice(0, 7);
    const gastoMes = transacoes.filter((t) => t.tipo === "Despesa" &&
t.data.startsWith(mesAtual)).reduce((s, t) => s + t.valor, 0);
    if (orcamentoMensal && gastoMes > orcamentoMensal) lista.push({ icone: Wallet,
texto: "Seus gastos do m\u00EAs j\u00E1 passaram do or\u00E7amento definido." });
    if (lista.length === 0) lista.push({ icone: Sparkles, texto: "Tudo em ordem por
aqui \u2014 aproveite o seu dia!" });
    return lista.slice(0, 4);
  }, [tarefas, habitos, metas, transacoes, orcamentoMensal]);
}
function TelaInicio({ irPara, onAbrirNotificacoes, onAbrirPerfil, qtdNotificacoes }) {
  const { perfil, tarefas, habitos, metas, transacoes, humorHoje, setHumorHoje,
alternarTarefa, alternarHabito } = useApp();
  const sugestoes = useSugestoes();
  const hora = (/* __PURE__ */ new Date()).getHours();
  const saudacao = hora < 12 ? "Bom dia" : hora < 18 ? "Boa tarde" : "Boa noite";
  const IconeHora = hora < 12 ? Sun : hora < 18 ? Sunset : MoonStar;
  const frase = FRASES[(/* __PURE__ */ new Date()).getDate() % FRASES.length];
  const tarefasHoje = tarefas.filter((t) => tarefaOcorreEm(t, HOJE_ISO));
  const concluidasHoje = tarefasHoje.filter((t) => tarefaConcluidaEm(t, HOJE_ISO));
  const habitosHoje = habitos;
  const habitosFeitos = habitos.filter((h) => h.historico[HOJE_ISO]);
  const totalItens = tarefasHoje.length + habitosHoje.length;
  const totalFeitos = concluidasHoje.length + habitosFeitos.length;
  const pctDia = totalItens ? totalFeitos / totalItens * 100 : 0;
  const proximosCompromissos = tarefasHoje.filter((t) => !tarefaConcluidaEm(t,
HOJE_ISO)).sort((a, b) => (a.hora || "").localeCompare(b.hora || "")).slice(0, 3);
  const mesAtual = HOJE_ISO.slice(0, 7);
  const entradasMes = transacoes.filter((t) => t.tipo === "Receita" &&
t.data.startsWith(mesAtual)).reduce((s, t) => s + t.valor, 0);
  const saidasMes = transacoes.filter((t) => t.tipo === "Despesa" &&
t.data.startsWith(mesAtual)).reduce((s, t) => s + t.valor, 0);
  const saldoGeral = transacoes.reduce((s, t) => s + (t.tipo === "Receita" ? t.valor :
-t.valor), 0);
  const metasEmAndamento = metas.filter((m) => m.progresso < 100).slice(0, 2);
  return (/* __PURE__ */ React.createElement("div", { style: { padding: "0 18px 90px"
} }, (/* __PURE__ */ React.createElement("div", { style: { display: "flex",
alignItems: "center", justifyContent: "space-between", paddingTop: 20 } }, (/*
__PURE__ */ React.createElement("div", { style: { display: "flex", alignItems:
"center", gap: 6 } }, (/* __PURE__ */ React.createElement(IconeHora, { size: 18,
color: C.champagne })), (/* __PURE__ */ React.createElement("h1", { style: { fontSize:
19, fontWeight: 700, color: C.texto, margin: 0 } }, saudacao, ", ", perfil?.nome)), (/*
__PURE__ */ React.createElement("div", { style: { display: "flex", gap: 8,
alignItems: "center" } }, (/* __PURE__ */ React.createElement("button", { onClick:
onAbrirNotificacoes, style: { position: "relative", background: C.rosaClaro, border:
"none", width: 34, height: 34, borderRadius: 17, display: "flex", alignItems:
"center", justifyContent: "center" } }, (/* __PURE__ */ React.createElement(Bell, {
size: 15, color: C.texto })), qtdNotificacoes > 0 && (/* __PURE__ */
React.createElement("span", { style: { position: "absolute", top: -2, right: -2,
width: 14, height: 14, borderRadius: 7, background: C.vermelho, color: "#fff",
fontSize: 8, display: "flex", alignItems: "center", justifyContent: "center" } },
qtdNotificacoes))), (/* __PURE__ */ React.createElement("button", { onClick:
onAbrirPerfil, style: { background: `linear-gradient(135deg, ${C.rosaCh\u00e1},
${C.lilas})`, border: "none", width: 34, height: 34, borderRadius: 17, display:
"flex", alignItems: "center", justifyContent: "center", fontSize: 15 } },
perfil?.avatar || "\u{1F338}"))), (/* __PURE__ */ React.createElement("p", { style: {
fontSize: 12, color: C.textoSuave, margin: "4px 0 2px" } },

```

```

formatarDataLonga(HOJE_ISO)), /* @__PURE__ */ React.createElement("p", { style: {
  fontSize: 13, color: C.texto, fontStyle: "italic", margin: "6px 0 0" } }, "'", frase,
  "'"), /* @__PURE__ */ React.createElement(Cartao, { style: { marginTop: 16, padding:
  18, display: "flex", alignItems: "center", gap: 16 } }, /* @__PURE__ */
  React.createElement(ProgressoCirculo, { pct: pctDia })), /* @__PURE__ */
  React.createElement("div", { style: { flex: 1 } }, /* @__PURE__ */
  React.createElement("p", { style: { fontSize: 13, fontWeight: 600, color: C.texto,
  margin: 0 } }, "Progresso de hoje"), /* @__PURE__ */ React.createElement("p", { style:
  { fontSize: 12, color: C.textoSuaVe, margin: "2px 0 8px" } }, totalFeitos, " de ",
  totalItens, " conclu\xEDdos"), /* @__PURE__ */ React.createElement("div", { style: {
  display: "flex", gap: 6 } }, HUMORES.map((h) => /* @__PURE__ */
  React.createElement("button", { key: h.valor, onClick: () => setHumorHoje(h.valor),
  style: { fontSize: 16, background: humorHoje === h.valor ? C.rosaClaro :
  "transparent", border: "none", borderRadius: 10, padding: 3 } }, h.emoji)))), /*
  @__PURE__ */ React.createElement("button", { onClick: () => irPara("dia"), style: {
  width: "100%", marginTop: 14, padding: "13px 0", borderRadius: 999, border: "none",
  color: "#fff", fontWeight: 600, fontSize: 13, background: `linear-gradient(135deg,
  ${C.rosaCh\u00e1}, ${C.lilas})` } }, "Organizar meu dia"), /* @__PURE__ */
  React.createElement(TituloSecao, null, "Pr\u00f3ximos compromissos"),
  proximosCompromissos.length === 0 ? /* @__PURE__ */ React.createElement(EstadoVazio, {
  texto: "Nenhum compromisso pendente hoje \u{1F389}" }) : /* @__PURE__ */
  React.createElement("div", { style: { display: "flex", flexDirection: "column", gap: 8
  } }, proximosCompromissos.map((t) => /* @__PURE__ */ React.createElement(
    ItemLinha,
    {
      key: t.id,
      icone: Clock,
      corIcone: CORES_PRIORIDADE[t.prioridade],
      titulo: t.titulo,
      subtitulo: `${t.hora} || "\u2014" \xB7 ${t.categoria}`,
      direita: /* @__PURE__ */ React.createElement("button", { onClick: () =>
        alternarTarefa(t.id, HOJE_ISO), style: { width: 26, height: 26, borderRadius: 13,
        border: `2px solid ${C.rosaClaro}`, background: "none" } })),
    }
  )), /* @__PURE__ */ React.createElement(TituloSecao, null, "H\u00e1bitos de hoje"),
  habitos.length === 0 ? /* @__PURE__ */ React.createElement(EstadoVazio, { texto:
  "Nenhum h\u00e1bito cadastrado ainda." }) : /* @__PURE__ */ React.createElement("div", {
  style: { display: "flex", flexDirection: "column", gap: 8 } }, habitos.map((h) => {
    const feito = !!h.historico[HOJE_ISO];
    return /* @__PURE__ */ React.createElement(
      ItemLinha,
      {
        key: h.id,
        icone: Repeat,
        corIcone: C.lilas,
        titulo: h.nome,
        subtitulo: `${h.frequencia} \xB7 sequ\u00eancia de ${sequenciaHabito(h)}
        dia(s)`,
        riscado: feito,
        direita: /* @__PURE__ */ React.createElement("button", { onClick: () =>
          alternarHabito(h.id, HOJE_ISO), style: { width: 26, height: 26, borderRadius: 13,
          border: "none", background: feito ? C.champagne : C.rosaClaro, display: "flex",
          alignItems: "center", justifyContent: "center" } }, feito && /* @__PURE__ */
          React.createElement(Check, { size: 14, color: "#fff" })))
      }
    );
  })), /* @__PURE__ */ React.createElement(TituloSecao, null, "Resumo financeiro"), /*
  @__PURE__ */ React.createElement(Cartao, { style: { padding: 16 } }, /* @__PURE__ */
  React.createElement("div", { style: { display: "flex", justifyContent: "space-
  between", marginBottom: 10 } }, /* @__PURE__ */ React.createElement("span", { style: {
  fontSize: 12, color: C.textoSuaVe } }, "Saldo dispon\xEDvel"), /* @__PURE__ */
  React.createElement("span", { style: { fontSize: 15, fontWeight: 700, color: C.texto }
  }, formatBRL(saldoGeral))), /* @__PURE__ */ React.createElement("div", { style: {
  display: "flex", gap: 10 } }, /* @__PURE__ */ React.createElement("div", { style: {
  flex: 1, display: "flex", alignItems: "center", gap: 6 } }, /* @__PURE__ */
  React.createElement(ArrowUpCircle, { size: 15, color: C.verde })), /* @__PURE__ */

```

```

React.createElement("span", { style: { fontSize: 12, color: C.textoSuaVe } },
formatBRL(entradasMes))), /* @__PURE__ */ React.createElement("div", { style: { flex:
1, display: "flex", alignItems: "center", gap: 6 } }, /* @__PURE__ */
React.createElement(ArrowDownCircle, { size: 15, color: C.vermelho })), /* @__PURE__ */
React.createElement("span", { style: { fontSize: 12, color: C.textoSuaVe } },
formatBRL(saidasMes)))))), /* @__PURE__ */ React.createElement(TituloSecao, null,
"Metas em andamento"), metasEmAndamento.length === 0 ? /* @__PURE__ */
React.createElement(EstadoVazio, { texto: "Nenhuma meta em andamento." }) : /*
@__PURE__ */ React.createElement("div", { style: { display: "flex", flexDirection:
"column", gap: 10 } }, metasEmAndamento.map((m) => /* @__PURE__ */
React.createElement(Cartao, { key: m.id, style: { padding: 14 } }, /* @__PURE__ */
React.createElement("div", { style: { display: "flex", justifyContent: "space-
between", marginBottom: 6 } }, /* @__PURE__ */ React.createElement("span", { style: {
fontSize: 13, fontWeight: 500, color: C.texto } }, m.titulo), /* @__PURE__ */
React.createElement("span", { style: { fontSize: 12, color: C.textoSuaVe } },
m.progresso, "%")), /* @__PURE__ */ React.createElement(ProgressoBarra, { pct:
m.progresso }))), /* @__PURE__ */ React.createElement(TituloSecao, null,
"Sugest\u00f5es para voc\u00ea"), /* @__PURE__ */ React.createElement("div", { style: {
display: "flex", flexDirection: "column", gap: 8 } }, sugestoes.map((s, i) => /*
@__PURE__ */ React.createElement("div", { key: i, style: { display: "flex",
alignItems: "center", gap: 10, background: `linear-gradient(135deg, ${C.rosaClaro},
${C.malva}33)`, borderRadius: 16, padding: 12 } }, /* @__PURE__ */
React.createElement(s.icone, { size: 16, color: C.texto })), /* @__PURE__ */
React.createElement("span", { style: { fontSize: 12.5, color: C.texto } },
s.texto))));
}
function sequenciaHabitto(h) {
  let seq = 0;
  for (let i = 0; i < 60; i++) {
    if (h.historico[isoDeHoje(-i)]) seq++;
    else break;
  }
  return seq;
}
function TelaMeuDia({ onAbrirNotificacoes, onAbrirPerfil, qtdNotificacoes }) {
  const { tarefas, adicionarTarefa, editarTarefa, removerTarefa, alternarTarefa } =
useApp();
  const [visao, setVisao] = useState("dia");
  const [dataSel, setDataSel] = useState(HOJE_ISO);
  const [modal, setModal] = useState(null);
  const tarefasDoDia = tarefas.filter((t) => tarefaOcorreEm(t, dataSel)).sort((a, b)
=> (a.hora || "").localeCompare(b.hora || ""));
  const concluidasDoDia = tarefasDoDia.filter((t) => tarefaConcluidaEm(t, dataSel));
  const pct = tarefasDoDia.length ? concluidasDoDia.length / tarefasDoDia.length * 100
: 0;
  const semana = Array.from({ length: 7 }, (_, i) => isoDeHoje(i -
indiceSemana(HOJE_ISO)));
  function mover(idx, dir) {
    const alvo = tarefasDoDia[idx];
    const outro = tarefasDoDia[idx + dir];
    if (!alvo || !outro) return;
    const h1 = alvo.hora, h2 = outro.hora;
    editarTarefa(alvo.id, { hora: h2 });
    editarTarefa(outro.id, { hora: h1 });
  }
  const campos = [
    { chave: "titulo", rotulo: "T\u00edtulo da tarefa", tipo: "texto", placeholder: "Ex:
Ligar para o dentista" },
    { chave: "categoria", rotulo: "Categoria", tipo: "select", opcoes:
CATEGORIAS_TAREFA.map((c) => ({ valor: c, rotulo: c })),
    { chave: "prioridade", rotulo: "Prioridade", tipo: "select", opcoes:
PRIORIDADES.map((c) => ({ valor: c, rotulo: c })),
    { chave: "hora", rotulo: "Hor\u00e1rio", tipo: "hora" },
    { chave: "data", rotulo: "Data", tipo: "data" },
    { chave: "recorrente", rotulo: "Tarefa recorrente", tipo: "toggle" },
    { chave: "frequencia", rotulo: "Repetir", tipo: "select", opcoes: [{ valor:

```

```

"diaria", rotulo: "Todo dia" }, { valor: "semanal", rotulo: "Dias da semana" } ] },
  { chave: "diasSemana", rotulo: "Quais dias", tipo: "multiselect", opcoes:
DIAS_SEMANA.map((d, i) => ({ valor: i, rotulo: d }))) },
  { chave: "lembrete", rotulo: "Criar lembrete", tipo: "toggle" }
];
return /* __PURE__ */ React.createElement("div", { style: { padding: "0 18px 90px"
} }, /* __PURE__ */ React.createElement(Cabecalho, { titulo: "Meu Dia", subtitulo:
formatarDataLonga(dataSel), onAbrirNotificacoes, onAbrirPerfil, qtdNotificacoes })), /*
__PURE__ */ React.createElement("div", { style: { margin: "0 0 14px", display:
"flex", gap: 8 } }, ["dia", "semana"].map((v) => /* __PURE__ */
React.createElement("button", { key: v, onClick: () => setVisao(v), style: { flex: 1,
padding: "8px 0", borderRadius: 999, border: "none", fontSize: 12, fontWeight: 600,
background: visao === v ? C.rosaClaro : C.rosaClaro, color: visao === v ? "#fff" : C.texto
} }, v === "dia" ? "Dia" : "Semana"))), visao === "semana" && /* __PURE__ */
React.createElement("div", { style: { display: "flex", gap: 8, overflowX: "auto",
paddingBottom: 8, marginBottom: 6 } }, semana.map((iso) => {
  const dt = tarefas.filter((t) => tarefaOcorreEm(t, iso));
  const dc = dt.filter((t) => tarefaConcluidaEm(t, iso));
  const p = dt.length ? dc.length / dt.length * 100 : 0;
  return /* __PURE__ */ React.createElement("button", { key: iso, onClick: () => {
    setDataSel(iso);
    setVisao("dia");
  }, style: { minWidth: 60, background: iso === dataSel ? C.rosaChamado : C.card,
border: "none", borderRadius: 16, padding: "10px 6px", boxShadow: "0 2px 8px
rgb(203,180,224,0.12)" } }, /* __PURE__ */ React.createElement("p", { style: {
fontSize: 10, color: iso === dataSel ? "#fff" : C.textoSuave, margin: 0,
textTransform: "capitalize" } }, diaCurto(iso)), /* __PURE__ */
React.createElement("p", { style: { fontSize: 14, fontWeight: 700, color: iso ===
dataSel ? "#fff" : C.texto, margin: "2px 0 6px" } }, numeroDia(iso)), /* __PURE__ */
React.createElement(ProgressoBarra, { pct: p, cor: iso === dataSel ? "#fff" :
C.champagne, alto: 4 })));
})), /* __PURE__ */ React.createElement(Cartao, { style: { padding: 14, display:
"flex", alignItems: "center", gap: 12, marginBottom: 14 } }, /* __PURE__ */
React.createElement(ProgressoCirculo, { pct, size: 48, stroke: 5 })), /* __PURE__ */
React.createElement("div", null, /* __PURE__ */ React.createElement("p", { style: {
fontSize: 13, fontWeight: 600, color: C.texto, margin: 0 } }, "Progresso do dia"), /*
__PURE__ */ React.createElement("p", { style: { fontSize: 12, color: C.textoSuave,
margin: "2px 0 0" } }, concluidasDoDia.length, " de ", tarefasDoDia.length, "
tarefas"))), tarefasDoDia.length === 0 ? /* __PURE__ */
React.createElement(EstadoVazio, { texto: "Nenhuma tarefa para este dia. Toque em +
para adicionar." }) : /* __PURE__ */ React.createElement("div", { style: { display:
"flex", flexDirection: "column", gap: 8 } }, tarefasDoDia.map((t, idx) => {
  const feita = tarefaConcluidaEm(t, dataSel);
  return /* __PURE__ */ React.createElement(Cartao, { key: t.id, style: { padding:
14, display: "flex", alignItems: "center", gap: 10 } }, /* __PURE__ */
React.createElement("button", { onClick: () => alternarTarefa(t.id, dataSel), style: {
width: 24, height: 24, borderRadius: 12, border: `2px solid
${CORES_PRIORIDADE[t.prioridade]}`, background: feita ? CORES_PRIORIDADE[t.prioridade]
: "none", display: "flex", alignItems: "center", justifyContent: "center", flexShrink:
0 } }, feita && /* __PURE__ */ React.createElement(Check, { size: 13, color: "#fff"
})), /* __PURE__ */ React.createElement("div", { style: { flex: 1, minWidth: 0,
onClick: () => setModal({ editando: t }) }, /* __PURE__ */ React.createElement("p", {
style: { fontSize: 13.5, fontWeight: 500, color: C.texto, margin: 0, textDecoratoin:
feita ? "line-through" : "none", opacity: feita ? 0.55 : 1 } }, t.titulo), /*
__PURE__ */ React.createElement("div", { style: { display: "flex", gap: 6, marginTop:
4, alignItems: "center" } }, /* __PURE__ */ React.createElement("span", { style: {
fontSize: 11, color: C.textoSuave } }, t.hora || "sem hora"), /* __PURE__ */
React.createElement(Pill, { cor: CORES_PRIORIDADE[t.prioridade] }, t.prioridade), /*
__PURE__ */ React.createElement(Pill, { cor: C.lilas }, t.categoria), t.recorrente &&
/* __PURE__ */ React.createElement(Repeat, { size: 11, color: C.textoSuave }},
t.lembrete && /* __PURE__ */ React.createElement(Bell, { size: 11, color:
C.textoSuave }))), /* __PURE__ */ React.createElement("div", { style: { display:
"flex", flexDirection: "column" } }, /* __PURE__ */ React.createElement(BotaoIcones, {
icone: ChevronLeft, onClick: () => mover(idx, -1) }, /* __PURE__ */
React.createElement(BotaoIcones, { icone: ChevronRight, onClick: () => mover(idx, 1)
})), /* __PURE__ */ React.createElement(BotaoIcones, { icone: Trash2, cor: C.vermelho,

```

```

onClick: () => removerTarefa(t.id) }));
    })), /* @__PURE__ */ React.createElement(BotaoFlutuante, { onClick: () => setModal({
editando: { data: dataSel, categoria: "Pessoal", prioridade: "M\u00E9dia", recorrente:
false, frequencia: "diaria", diasSemana: [], lembrete: false } })), modal && /*
@__PURE__ */ React.createElement(
  ModalFormulario,
  {
    titulo: modal.editando.id ? "Editar tarefa" : "Nova tarefa",
    campos,
    valores: modal.editando,
    corDestaque: C.rosaChamada,
    onFechar: () => setModal(null),
    onExcluir: modal.editando.id ? () => {
      removerTarefa(modal.editando.id);
      setModal(null);
    } : void 0,
    onSalvar: (dados) => {
      if (!dados.titulo) return;
      if (modal.editando.id) editarTarefa(modal.editando.id, dados);
      else adicionarTarefa(dados);
      setModal(null);
    }
  }
));
}
function TelaVida({ onAbrirNotificacoes, onAbrirPerfil, qtdNotificacoes }) {
  const [sub, setSub] = useState("metas");
  const subs = [
    { id: "metas", label: "Metas" },
    { id: "habitos", label: "H\u00e1bitos" },
    { id: "autocuidado", label: "Autocuidado" },
    { id: "diario", label: "Di\u00e1rio" },
    { id: "desejos", label: "Desejos" },
    { id: "projetos", label: "Projetos" }
  ];
  return /* @__PURE__ */ React.createElement("div", { style: { padding: "0 18px 90px"
} }, /* @__PURE__ */ React.createElement(Cabecalho, { titulo: "Vida", subtitulo: "Sua
organiza\u00e7\u00e3o pessoal", onAbrirNotificacoes, onAbrirPerfil, qtdNotificacoes })), /*
@__PURE__ */ React.createElement("div", { style: { display: "flex", gap: 6, overflowX:
"auto", paddingBottom: 4 } }, subs.map((s) => /* @__PURE__ */
React.createElement("button", { key: s.id, onClick: () => setSub(s.id), style: {
whiteSpace: "nowrap", padding: "7px 13px", borderRadius: 999, border: "none",
fontSize: 11.5, fontWeight: 600, background: sub === s.id ? C.texto : C.rosaClaro,
color: sub === s.id ? "#fff" : C.texto } }, s.label))), /* @__PURE__ */
React.createElement("div", { style: { marginTop: 14 } }, sub === "metas" && /*
@__PURE__ */ React.createElement(SecaoMetas, null), sub === "habitos" && /* @__PURE__
*/ React.createElement(SecaoHabitos, null), sub === "autocuidado" && /* @__PURE__ */
React.createElement(SecaoAutocuidado, null), sub === "diario" && /* @__PURE__ */
React.createElement(SecaoDiario, null), sub === "desejos" && /* @__PURE__ */
React.createElement(SecaoDesejos, null), sub === "projetos" && /* @__PURE__ */
React.createElement(SecaoProjetos, null)));
}
function SecaoMetas() {
  const { metas, adicionarMeta, editarMeta, removerMeta } = useApp();
  const [modal, setModal] = useState(null);
  const campos = [
    { chave: "titulo", rotulo: "T\u00edtulo da meta", tipo: "texto", placeholder: "Ex:
Ler 12 livros este ano" },
    { chave: "categoria", rotulo: "Categoria", tipo: "select", opcoes:
CATEGORIAS_META.map((c) => ({ valor: c, rotulo: c })),
    { chave: "prazo", rotulo: "Prazo", tipo: "data" },
    { chave: "progresso", rotulo: "Progresso (%)", tipo: "numero", placeholder: "0 a
100" }
  ];
  return /* @__PURE__ */ React.createElement("div", { style: { display: "flex",
flexDirection: "column", gap: 10 } }, metas.length === 0 && /* @__PURE__ */

```

```

React.createElement(EstadoVazio, { texto: "Nenhuma meta cadastrada." })), metas.map((m)
=> /* __PURE__ */ React.createElement(Cartao, { key: m.id, style: { padding: 14 },
onClick: () => setModal({ editando: m }) }}, /* __PURE__ */ React.createElement("div",
{ style: { display: "flex", justifyContent: "space-between", marginBottom: 6 } }, /*
__PURE__ */ React.createElement("span", { style: { fontSize: 13.5, fontWeight: 500,
color: C.texto } }, m.titulo), /* __PURE__ */ React.createElement(Pill, { cor:
C.lilas }, m.categoria)), /* __PURE__ */ React.createElement(ProgressoBarra, { pct:
m.progresso })), /* __PURE__ */ React.createElement("div", { style: { display: "flex",
justifyContent: "space-between", marginTop: 6 } }, /* __PURE__ */
React.createElement("span", { style: { fontSize: 11, color: C.textoSuaVe } }, "Prazo:
", formatarDataLonga(m.prazo)), /* __PURE__ */ React.createElement("span", { style: {
fontSize: 11, color: C.textoSuaVe } }, m.progresso, "%))), /* __PURE__ */
React.createElement(BotaoFlutuante, { onClick: () => setModal({ editando: { categoria:
"Pessoal", progresso: 0, prazo: isoDeHoje(30) } }) })), modal && /* __PURE__ */
React.createElement(
  ModalFormulario,
  {
    titulo: modal.editando.id ? "Editar meta" : "Nova meta",
    campos,
    valores: modal.editando,
    corDestaque: C.lilas,
    onFechar: () => setModal(null),
    onExcluir: modal.editando.id ? () => {
      removerMeta(modal.editando.id);
      setModal(null);
    } : void 0,
    onSalvar: (d) => {
      if (!d.titulo) return;
      modal.editando.id ? editarMeta(modal.editando.id, d) : adicionarMeta(d);
      setModal(null);
    }
  }
));
}
function SecaoHabitos() {
  const { habitos, adicionarHabitto, editarHabitto, removerHabitto, alternarHabitto } =
  useApp();
  const [modal, setModal] = useState(null);
  const campos = [
    { chave: "nome", rotulo: "Nome do h\xE1bito", tipo: "texto", placeholder: "Ex:
Meditar" },
    { chave: "frequencia", rotulo: "Frequ\xEancia", tipo: "select", opcoes: [{ valor:
"Di\xE1rio", rotulo: "Di\xE1rio" }, { valor: "Semanal", rotulo: "Semanal" } ] },
    { chave: "lembrete", rotulo: "Criar lembrete", tipo: "toggle" }
  ];
  return /* __PURE__ */ React.createElement("div", { style: { display: "flex",
flexDirection: "column", gap: 10 } }, habitos.length === 0 && /* __PURE__ */
React.createElement(EstadoVazio, { texto: "Nenhum h\xE1bito cadastrado." })),
habitos.map((h) => {
  const feito = !!h.historico[HOJE_ISO];
  const ultimosDias = Array.from({ length: 7 }, (_, i) => isoDeHoje(-6 + i));
  return /* __PURE__ */ React.createElement(Cartao, { key: h.id, style: { padding:
14 } }, /* __PURE__ */ React.createElement("div", { style: { display: "flex",
alignItems: "center", justifyContent: "space-between" } }, /* __PURE__ */
React.createElement("div", { onClick: () => setModal({ editando: h }), style: { flex:
1 } }, /* __PURE__ */ React.createElement("p", { style: { fontSize: 13.5, fontWeight:
500, color: C.texto, margin: 0 } }, h.nome), /* __PURE__ */
React.createElement("div", { style: { display: "flex", gap: 6, alignItems: "center",
marginTop: 4 } }, /* __PURE__ */ React.createElement(Pill, { cor: C.lilas },
h.frequencia), /* __PURE__ */ React.createElement("span", { style: { fontSize: 11,
color: C.textoSuaVe, display: "flex", alignItems: "center", gap: 3 } }, /* __PURE__
*/ React.createElement(Flame, { size: 11, color: C.champagne })), " ",
sequenciaHabitto(h), " dias))), /* __PURE__ */ React.createElement("button", {
onClick: () => alternarHabitto(h.id, HOJE_ISO), style: { width: 30, height: 30,
borderRadius: 15, border: "none", background: feito ? C.champagne : C.rosaClaro,
display: "flex", alignItems: "center", justifyContent: "center" } }, feito && /*

```

```

    __PURE__ */ React.createElement(Check, { size: 15, color: "#fff" }))), /* __PURE__
    */ React.createElement("div", { style: { display: "flex", gap: 4, marginTop: 10 } },
    ultimosDias.map((iso) => /* __PURE__ */ React.createElement("div", { key: iso, style:
    { flex: 1, height: 6, borderRadius: 3, background: h.historico[iso] ? C.champagne :
    C.rosaClaro } }))),);
    }, /* __PURE__ */ React.createElement(BotaoFlutuante, { onClick: () => setModal({
    editando: { frequencia: "Di\xE1rio", lembrete: false } } })), modal && /* __PURE__ */
    React.createElement(
      ModalFormulario,
      {
        titulo: modal.editando.id ? "Editar h\xE1bito" : "Novo h\xE1bito",
        campos,
        valores: modal.editando,
        corDestaque: C.lilas,
        onFechar: () => setModal(null),
        onExcluir: modal.editando.id ? () => {
          removerHabito(modal.editando.id);
          setModal(null);
        } : void 0,
        onSalvar: (d) => {
          if (!d.nome) return;
          modal.editando.id ? editarHabito(modal.editando.id, d) : adicionarHabito(d);
          setModal(null);
        }
      }
    ));
  }
}
function SecaoAutocuidado() {
  const { autocuidado, adicionarAutocuidado, alternarAutocuidado, removerAutocuidado }
  = useApp();
  const [modal, setModal] = useState(null);
  const feitos = autocuidado.filter((a) => a.feitoHoje).length;
  return /* __PURE__ */ React.createElement("div", null, /* __PURE__ */
  React.createElement(Cartao, { style: { padding: 14, marginBottom: 12 } }, /* __PURE__
  */ React.createElement("p", { style: { fontSize: 13, fontWeight: 600, color: C.texto,
  margin: "0 0 8px" } }, "Checklist de hoje"), /* __PURE__ */
  React.createElement(ProgressoBarra, { pct: autocuidado.length ? feitos /
  autocuidado.length * 100 : 0 })), /* __PURE__ */ React.createElement("div", { style:
  { display: "flex", flexDirection: "column", gap: 8 } }, autocuidado.length === 0 && /*
  __PURE__ */ React.createElement(EstadoVazio, { texto: "Nenhum item de autocuidado
  ainda." })), autocuidado.map((a) => /* __PURE__ */ React.createElement(
    ItemLinha,
    {
      key: a.id,
      icone: Sparkles,
      corIcone: C.malva,
      titulo: a.nome,
      riscado: a.feitoHoje,
      direita: /* __PURE__ */ React.createElement("div", { style: { display: "flex",
      gap: 6 } }, /* __PURE__ */ React.createElement("button", { onClick: () =>
      alternarAutocuidado(a.id), style: { width: 26, height: 26, borderRadius: 13, border:
      "none", background: a.feitoHoje ? C.champagne : C.rosaClaro, display: "flex",
      alignItems: "center", justifyContent: "center" } }, a.feitoHoje && /* __PURE__ */
      React.createElement(Check, { size: 13, color: "#fff" })), /* __PURE__ */
      React.createElement(BotaoIcone, { icone: Trash2, cor: C.vermelho, onClick: () =>
      removerAutocuidado(a.id) })))
    }
  )), /* __PURE__ */ React.createElement(BotaoFlutuante, { onClick: () => setModal({
  editando: {} } })), modal && /* __PURE__ */ React.createElement(
    ModalFormulario,
    {
      titulo: "Novo item de autocuidado",
      campos: [{ chave: "nome", rotulo: "Nome", tipo: "texto", placeholder: "Ex:
      Hidratar a pele" }],
      valores: modal.editando,
      corDestaque: C.malva,

```

```

        onFechar: () => setModal(null),
        onSalvar: (d) => {
            if (!d.nome) return;
            adicionarAutocuidado(d);
            setModal(null);
        }
    }
    ));
}

function SecaoDiario() {
    const { diario, adicionarDiario, removerDiario } = useApp();
    const [modal, setModal] = useState(null);
    const campos = [
        { chave: "texto", rotulo: "Como foi o seu dia?", tipo: "textarea", placeholder:
"Escreva livremente..." },
        { chave: "humor", rotulo: "Humor", tipo: "select", opcoes: HUMORES.map((h) => ({
valor: h.valor, rotulo: `${h.emoji} ${h.rotulo}` }))) }
    ];
    return /* __PURE__ */ React.createElement("div", { style: { display: "flex",
flexDirection: "column", gap: 10 } }, diario.length === 0 && /* __PURE__ */
React.createElement(EstadoVazio, { texto: "Nenhum registro no di\u00e1rio ainda." })),
[...diario].reverse().map((d) => {
    const h = HUMORES.find((x) => x.valor === d.humor);
    return /* __PURE__ */ React.createElement(Cartao, { key: d.id, style: { padding:
14 } }, /* __PURE__ */ React.createElement("div", { style: { display: "flex",
justifyContent: "space-between", marginBottom: 6 } }, /* __PURE__ */
React.createElement("span", { style: { fontSize: 12, color: C.textoSuave } },
formatarDataLonga(d.data)), /* __PURE__ */ React.createElement("div", { style: {
display: "flex", gap: 8, alignItems: "center" } }, /* __PURE__ */
React.createElement("span", { style: { fontSize: 16 } }, h?.emoji), /* __PURE__ */
React.createElement(BotaoIcone, { icone: Trash2, cor: C.vermelho, onClick: () =>
removerDiario(d.id) }))), /* __PURE__ */ React.createElement("p", { style: {
fontSize: 13, color: C.texto, margin: 0, lineHeight: 1.5 } }, d.texto));
    } ), /* __PURE__ */ React.createElement(BotaoFlutuante, { onClick: () => setModal({
editando: { humor: "bem" } } })), modal && /* __PURE__ */ React.createElement(
ModalFormulario,
{
    titulo: "Novo registro",
    campos,
    valores: modal.editando,
    corDestaque: C.rosaChamada,
    onFechar: () => setModal(null),
    onSalvar: (d) => {
        if (!d.texto) return;
        adicionarDiario({ ...d, data: HOJE_ISO });
        setModal(null);
    }
}
));
}

function SecaoDesejos() {
    const { desejos, adicionarDesejo, removerDesejo } = useApp();
    const [modal, setModal] = useState(null);
    const campos = [
        { chave: "titulo", rotulo: "O que voc\u00ea deseja?", tipo: "texto", placeholder:
"Ex: Curso de fotografia" },
        { chave: "categoria", rotulo: "Categoria", tipo: "texto", placeholder: "Ex: Lazer"
    },
        { chave: "preco", rotulo: "Valor estimado (R$)", tipo: "numero" }
    ];
    return /* __PURE__ */ React.createElement("div", { style: { display: "flex",
flexDirection: "column", gap: 10 } }, desejos.length === 0 && /* __PURE__ */
React.createElement(EstadoVazio, { texto: "Sua lista de desejos est\u00e1 vazia." })),
desejos.map((d) => /* __PURE__ */ React.createElement(
ItemLinha,
{

```

```

        key: d.id,
        icone: Gift,
        corIcone: C.champagne,
        titulo: d.titulo,
        subtítulo: `${d.categoria} \xB7 ${formatBRL(d.preco)}`,
        direita: /* __PURE__ */ React.createElement(BotaoIcone, { icone: Trash2, cor:
C.vermelho, onClick: () => removerDesejo(d.id) })
    }
    ), /* __PURE__ */ React.createElement(BotaoFlutuante, { onClick: () => setModal({
editando: {} }) })), modal && /* __PURE__ */ React.createElement(
    ModalFormulario,
    {
        titulo: "Novo desejo",
        campos,
        valores: modal.editando,
        corDestaque: C.champagne,
        onFechar: () => setModal(null),
        onSalvar: (d) => {
            if (!d.titulo) return;
            adicionarDesejo(d);
            setModal(null);
        }
    }
    ));
}
function SecaoProjetos() {
    const { projetos, adicionarProjeto, editarProjeto, removerProjeto } = useApp();
    const [modal, setModal] = useState(null);
    const campos = [
        { chave: "titulo", rotulo: "Nome do projeto", tipo: "texto", placeholder: "Ex:
Reforma do quarto" },
        { chave: "etapasTotal", rotulo: "Total de etapas", tipo: "numero" },
        { chave: "etapasConcluidas", rotulo: "Etapas conclu\xEDdas", tipo: "numero" }
    ];
    return /* __PURE__ */ React.createElement("div", { style: { display: "flex",
flexDirection: "column", gap: 10 } }, projetos.length === 0 && /* __PURE__ */
React.createElement(EstadoVazio, { texto: "Nenhum projeto pessoal ainda." })),
    projetos.map((p) => /* __PURE__ */ React.createElement(Cartao, { key: p.id, style: {
padding: 14 }, onClick: () => setModal({ editando: p }) }, /* __PURE__ */
React.createElement("div", { style: { display: "flex", alignItems: "center", gap: 10,
marginBottom: 8 } }, /* __PURE__ */ React.createElement(FolderKanban, { size: 16,
color: C.lilas })), /* __PURE__ */ React.createElement("span", { style: { fontSize:
13.5, fontWeight: 500, color: C.texto } }, p.titulo)), /* __PURE__ */
React.createElement(ProgressoBarra, { pct: p.etapasConcluidas / (p.etapasTotal || 1) *
100, cor: C.lilas })), /* __PURE__ */ React.createElement("p", { style: { fontSize:
11, color: C.textoSuaVe, margin: "6px 0 0" } }, p.etapasConcluidas, " de ",
p.etapasTotal, " etapas")), /* __PURE__ */ React.createElement(BotaoFlutuante, {
onClick: () => setModal({ editando: { etapasTotal: 1, etapasConcluidas: 0 } }) })),
    modal && /* __PURE__ */ React.createElement(
        ModalFormulario,
        {
            titulo: modal.editando.id ? "Editar projeto" : "Novo projeto",
            campos,
            valores: modal.editando,
            corDestaque: C.lilas,
            onFechar: () => setModal(null),
            onExcluir: modal.editando.id ? () => {
                removerProjeto(modal.editando.id);
                setModal(null);
            } : void 0,
            onSalvar: (d) => {
                if (!d.titulo) return;
                modal.editando.id ? editarProjeto(modal.editando.id, d) : adicionarProjeto(d);
                setModal(null);
            }
        }
    )
}

```

```

    ));
}
function TelaDinheiro({ onAbrirNotificacoes, onAbrirPerfil, qtdNotificacoes }) {
  const { transacoes, adicionarTransacao, removerTransacao, orcamentoMensal,
    setOrcamentoMensal, metasFinanceiras, adicionarMetaFinanceira,
    contribuirMetaFinanceira, removerMetaFinanceira } = useApp();
  const [modalTx, setModalTx] = useState(null);
  const [modalMeta, setModalMeta] = useState(null);
  const [editandoOrcamento, setEditandoOrcamento] = useState(false);
  const [novoOrcamento, setNovoOrcamento] = useState(orcamentoMensal);
  const mesAtual = HOJE_ISO.slice(0, 7);
  const doMes = transacoes.filter((t) => t.data.startsWith(mesAtual));
  const entradas = doMes.filter((t) => t.tipo === "Receita").reduce((s, t) => s +
    t.valor, 0);
  const saidas = doMes.filter((t) => t.tipo === "Despesa").reduce((s, t) => s +
    t.valor, 0);
  const saldo = transacoes.reduce((s, t) => s + (t.tipo === "Receita" ? t.valor : -
    t.valor), 0);
  const porCategoria = {};
  doMes.filter((t) => t.tipo === "Despesa").forEach((t) => {
    porCategoria[t.categoria] = (porCategoria[t.categoria] || 0) + t.valor;
  });
  const dadosGrafico = Object.entries(porCategoria).map(([cat, valor]) => ({ cat,
    valor }));
  const campos = [
    { chave: "tipo", rotulo: "Tipo", tipo: "select", opcoes: [{ valor: "Receita",
      rotulo: "Receita" }, { valor: "Despesa", rotulo: "Despesa" } ] },
    { chave: "descricao", rotulo: "Descrição", tipo: "texto", placeholder: "Ex:
      Supermercado" },
    { chave: "categoria", rotulo: "Categoria", tipo: "select", opcoes:
      (modalTx?.editando?.tipo === "Receita" ? CATEGORIAS_RECEITA :
      CATEGORIAS_DESPESA).map((c) => ({ valor: c, rotulo: c })),
    { chave: "conta", rotulo: "Conta", tipo: "select", opcoes: CONTAS.map((c) => ({
      valor: c, rotulo: c })),
    { chave: "valor", rotulo: "Valor (R$)", tipo: "numero" },
    { chave: "data", rotulo: "Data", tipo: "data" }
  ];
  const camposMeta = [
    { chave: "titulo", rotulo: "Nome da meta", tipo: "texto", placeholder: "Ex: Viagem
      dos sonhos" },
    { chave: "valorAlvo", rotulo: "Valor alvo (R$)", tipo: "numero" },
    { chave: "valorAtual", rotulo: "Valor já guardado (R$)", tipo: "numero" },
    { chave: "prazo", rotulo: "Prazo", tipo: "data" }
  ];
  return /* #__PURE__ */ React.createElement("div", { style: { padding: "0 18px 90px"
  } }, /* #__PURE__ */ React.createElement(Cabecalho, { titulo: "Dinheiro", subtítulo:
    "Seu controle financeiro", onAbrirNotificacoes, onAbrirPerfil, qtdNotificacoes }), /*
    #__PURE__ */ React.createElement(Cartao, { style: { padding: 18, background: `linear-
    gradient(135deg, ${C.rosaChá}, ${C.lilas})` } }, /* #__PURE__ */
    React.createElement("p", { style: { fontSize: 12, color: "#fff9", margin: 0 } },
    "Saldo disponível"), /* #__PURE__ */ React.createElement("p", { style: { fontSize:
    24, fontWeight: 700, color: "#fff", margin: "4px 0 12px" } }, formatBRL(saldo)), /*
    #__PURE__ */ React.createElement("div", { style: { display: "flex", gap: 16 } }, /*
    #__PURE__ */ React.createElement("div", { style: { display: "flex", gap: 5,
    alignItems: "center" } }, /* #__PURE__ */ React.createElement(ArrowUpCircle, { size:
    15, color: "#fff" })), /* #__PURE__ */ React.createElement("span", { style: { fontSize:
    12, color: "#fff" } }, formatBRL(entradas))), /* #__PURE__ */
    React.createElement("div", { style: { display: "flex", gap: 5, alignItems: "center" }
    }, /* #__PURE__ */ React.createElement(ArrowDownCircle, { size: 15, color: "#fff" })),
    /* #__PURE__ */ React.createElement("span", { style: { fontSize: 12, color: "#fff" }
    }, formatBRL(saidas))))) , /* #__PURE__ */ React.createElement(TituloSecao, { acao: /*
    #__PURE__ */ React.createElement("button", { onClick: () =>
    setEditandoOrcamento(true), style: { background: "none", border: "none", color:
    C.champagne, fontSize: 12, fontWeight: 600 } }, "Editar" ) }, "Orçamento mensal"),
    /* #__PURE__ */ React.createElement(Cartao, { style: { padding: 14 } },
    editandoOrcamento ? /* #__PURE__ */ React.createElement("div", { style: { display:

```

```

"flex", gap: 8 } }, /* @__PURE__ */ React.createElement("input", { type: "number",
style: estiloInput, value: novoOrcamento, onChange: (e) =>
setNovoOrcamento(Number(e.target.value)) }}, /* @__PURE__ */
React.createElement("button", { onClick: () => {
  setOrcamentoMensal(novoOrcamento);
  setEditandoOrcamento(false);
}, style: { background: C.champagne, border: "none", borderRadius: 12, padding: "0
16px", color: "#fff", fontSize: 12, fontWeight: 600 } }, "OK")) : /* @__PURE__ */
React.createElement(React.Fragment, null, /* @__PURE__ */ React.createElement("div", {
style: { display: "flex", justifyContent: "space-between", marginBottom: 6 } }, /*
@__PURE__ */ React.createElement("span", { style: { fontSize: 12, color: C.textoSuave
} }, "Gasto: ", formatBRL(saidas)), /* @__PURE__ */ React.createElement("span", {
style: { fontSize: 12, color: C.textoSuave } }, "Limite: ",
formatBRL(orcamentoMensal))), /* @__PURE__ */ React.createElement(ProgressoBarra, {
pct: orcamentoMensal ? saidas / orcamentoMensal * 100 : 0, cor: saidas >
orcamentoMensal ? C.vermelho : C.champagne })), dadosGrafico.length > 0 && /*
@__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__ */
React.createElement(TituloSecao, null, "Despesas por categoria"), /* @__PURE__ */
React.createElement(Cartao, { style: { padding: 14 } }, /* @__PURE__ */
React.createElement(ResponsiveContainer, { width: "100%", height: 160 }, /* @__PURE__
*/ React.createElement(BarChart, { data: dadosGrafico }, /* @__PURE__ */
React.createElement(CartesianGrid, { stroke: C.rosaClaro, vertical: false })), /*
@__PURE__ */ React.createElement(XAxis, { dataKey: "cat", tick: { fontSize: 9, fill:
C.textoSuave }, axisLine: false, tickLine: false, interval: 0, angle: -20, textAnchor:
"end", height: 50 })), /* @__PURE__ */ React.createElement(YAxis, { hide: true })), /*
@__PURE__ */ React.createElement(Tooltip, { contentStyle: { borderRadius: 12, border:
"none", fontSize: 12 }, formatter: (v) => formatBRL(v) })), /* @__PURE__ */
React.createElement(Bar, { dataKey: "valor", fill: C.rosaChá, radius: [6, 6, 0, 0]
}))), /* @__PURE__ */ React.createElement(TituloSecao, null, "Metas financeiras"),
/* @__PURE__ */ React.createElement("div", { style: { display: "flex", flexDirection:
"column", gap: 10 } }, metasFinanceiras.length === 0 && /* @__PURE__ */
React.createElement(EstadoVazio, { texto: "Nenhuma meta financeira ainda." })),
metasFinanceiras.map((m) => /* @__PURE__ */ React.createElement(Cartao, { key: m.id,
style: { padding: 14 } }, /* @__PURE__ */ React.createElement("div", { style: {
display: "flex", justifyContent: "space-between", marginBottom: 6 } }, /* @__PURE__ */
React.createElement("span", { style: { fontSize: 13, fontWeight: 500, color: C.texto }
}, m.titulo), /* @__PURE__ */ React.createElement(BotaoIcone, { icone: Trash2, cor:
C.vermelho, onClick: () => removerMetaFinanceira(m.id) })), /* @__PURE__ */
React.createElement(ProgressoBarra, { pct: m.valorAtual / (m.valorAlvo || 1) * 100,
cor: C.verde })), /* @__PURE__ */ React.createElement("div", { style: { display:
"flex", justifyContent: "space-between", alignItems: "center", marginTop: 8 } }, /*
@__PURE__ */ React.createElement("span", { style: { fontSize: 11, color: C.textoSuave
} }, formatBRL(m.valorAtual), " de ", formatBRL(m.valorAlvo)), /* @__PURE__ */
React.createElement("button", { onClick: () => contribuirMetaFinanceira(m.id, 50),
style: { background: C.verde, border: "none", borderRadius: 999, padding: "5px 12px",
color: "#fff", fontSize: 11, fontWeight: 600 } }, "+ R$50"))), /* @__PURE__ */
React.createElement("button", { onClick: () => setModalMeta({ editando: { valorAtual:
0, valorAlvo: 1e3, prazo: isoDeHoje(90) } })), style: { padding: "10px 0",
borderRadius: 14, border: `1.5px dashed ${C.lilas}`, background: "none", color:
C.lilas, fontSize: 12, fontWeight: 600 } }, "+ Nova meta financeira")), /* @__PURE__
*/ React.createElement(TituloSecao, null, "Hist\u00f3rico de movimentações"), /*
@__PURE__ */ React.createElement("div", { style: { display: "flex", flexDirection:
"column", gap: 8 } }, [...transacoes].sort((a, b) =>
b.data.localeCompare(a.data)).map((t) => /* @__PURE__ */ React.createElement(
ItemLinha,
{
  key: t.id,
  icone: t.tipo === "Receita" ? ArrowUpCircle : ArrowDownCircle,
  corIcone: t.tipo === "Receita" ? C.verde : C.vermelho,
  titulo: t.descricao,
  subtitulo: `${t.categoria} \xB7 ${t.conta} \xB7 ${/* @__PURE__ */ new
Date(t.data + "T12:00:00").toLocaleDateString("pt-BR")}`,
  direita: /* @__PURE__ */ React.createElement("div", { style: { display: "flex",
alignItems: "center", gap: 8 } }, /* @__PURE__ */ React.createElement("span", { style:
{ fontSize: 13, fontWeight: 600, color: t.tipo === "Receita" ? C.verde : C.vermelho }
}, t.tipo === "Receita" ? "+" : "-", formatBRL(t.valor)), /* @__PURE__ */

```

```

React.createElement(BotaoIcones, { icone: Trash2, cor: C.vermelho, onClick: () =>
  removerTransacao(t.id) })
}
)), /* #__PURE__ */ React.createElement(BotaoFlutuante, { onClick: () =>
  setModalTx({ editando: { tipo: "Despesa", categoria: CATEGORIAS_DESPESA[0], conta:
    CONTAS[0], data: HOJE_ISO } }) }), modalTx && /* #__PURE__ */ React.createElement(
  ModalFormulario,
  {
    titulo: "Nova movimentação",
    campos,
    valores: modalTx.editando,
    corDestaque: C.champagne,
    onFechar: () => setModalTx(null),
    onSalvar: (d) => {
      if (!d.descricao || !d.valor) return;
      adicionarTransacao(d);
      setModalTx(null);
    }
  }
), modalMeta && /* #__PURE__ */ React.createElement(
  ModalFormulario,
  {
    titulo: "Nova meta financeira",
    campos: camposMeta,
    valores: modalMeta.editando,
    corDestaque: C.verde,
    onFechar: () => setModalMeta(null),
    onSalvar: (d) => {
      if (!d.titulo) return;
      adicionarMetaFinanceira(d);
      setModalMeta(null);
    }
  }
));
}
function CelulaDiaEscala({ numero, tipoInfo, hoje, onClick }) {
  return /* #__PURE__ */ React.createElement(
    "button",
    {
      onClick,
      style: {
        aspectRatio: "1",
        border: hoje ? `2px solid ${C.rosaChá}` : "1px solid transparent",
        borderRadius: 14,
        background: tipoInfo ? tipoInfo.corFundo : C.card,
        display: "flex",
        flexDirection: "column",
        alignItems: "center",
        justifyContent: "center",
        gap: 2,
        cursor: "pointer",
        padding: 0
      }
    },
    /* #__PURE__ */ React.createElement("span", { style: { fontSize: 13, fontWeight:
      600, color: C.texto } }, numero),
    tipoInfo && /* #__PURE__ */ React.createElement("span", { style: { fontSize: 12,
      lineHeight: 1 } }, tipoInfo.emoji)
  );
}
const MISSOES_DIARIAS = [
  { id: "m1", texto: "Beber água" },
  { id: "m2", texto: "Comer uma fruta" },
  { id: "m3", texto: "Comer vegetais" },
  { id: "m4", texto: "Caminhar 10 min" },
  { id: "m5", texto: "Alongar o corpo" },

```

```

    { id: "m6", texto: "Medita\uE7\uE3o 5 min" },
    { id: "m7", texto: "Ler 10 min" },
    { id: "m8", texto: "Dormir 7h+" },
    { id: "m9", texto: "Evitar a\uE7\uFACar" },
    { id: "m10", texto: "Evitar fritura" },
    { id: "m11", texto: "Escovar dentes" },
    { id: "m12", texto: "Lavar o rosto" },
    { id: "m13", texto: "Passar protetor" },
    { id: "m14", texto: "Arrumar a cama" },
    { id: "m15", texto: "Anotar gratid\uE3o" }
  ];
  const estiloBotaoRedondo = { width: 34, height: 34, borderRadius: 17, border: "none",
  background: C.rosaClaro, color: C.texto, fontSize: 18, fontWeight: 600, cursor:
  "pointer" };
  function ContadorDiario({ emoji, titulo, valor, meta, unidade, corBarra, onSubtrair,
  onSomar }) {
    return /* __PURE__ */ React.createElement(
      Cartao,
      { style: { padding: 16, marginBottom: 12 } },
      /* __PURE__ */ React.createElement(
        "div",
        { style: { display: "flex", alignItems: "center", gap: 10, marginBottom: 10 } },
        /* __PURE__ */ React.createElement("span", { style: { fontSize: 22 } }, emoji),
        /* __PURE__ */ React.createElement(
          "div",
          null,
          /* __PURE__ */ React.createElement("p", { style: { fontSize: 15, fontWeight:
          600, color: C.texto, margin: 0 } }, titulo),
          /* __PURE__ */ React.createElement("p", { style: { fontSize: 12, color:
          C.textoSuaVe, margin: "2px 0 0" } }, `${valor}/${meta} ${unidade}`)
        ),
        /* __PURE__ */ React.createElement(ProgressoBarra, { pct: valor / meta * 100,
        cor: corBarra }),
        /* __PURE__ */ React.createElement(
          "div",
          { style: { display: "flex", justifyContent: "center", alignItems: "center",
          marginTop: 12 } },
          /* __PURE__ */ React.createElement(
            "div",
            { style: { display: "flex", gap: 10 } },
            /* __PURE__ */ React.createElement("button", { onClick: onSubtrair, style:
            estiloBotaoRedondo }, "\u2212"),
            /* __PURE__ */ React.createElement("button", { onClick: onSomar, style:
            estiloBotaoRedondo }, "+")
          ),
          /* __PURE__ */ React.createElement("div", { style: { display: "flex", gap: 10 } },
            /* __PURE__ */ React.createElement("button", { onClick: onSubtrair, style:
            estiloBotaoRedondo }, "\u2212"),
            /* __PURE__ */ React.createElement("button", { onClick: onSomar, style:
            estiloBotaoRedondo }, "+")
          )
        ),
        /* __PURE__ */ React.createElement("div", { style: { display: "flex", gap: 10 } },
          /* __PURE__ */ React.createElement("button", { onClick: onSubtrair, style:
          estiloBotaoRedondo }, "\u2212"),
          /* __PURE__ */ React.createElement("button", { onClick: onSomar, style:
          estiloBotaoRedondo }, "+")
        )
      ),
      /* __PURE__ */ React.createElement("div", { style: { display: "flex", gap: 10 } },
        /* __PURE__ */ React.createElement("button", { onClick: onSubtrair, style:
        estiloBotaoRedondo }, "\u2212"),
        /* __PURE__ */ React.createElement("button", { onClick: onSomar, style:
        estiloBotaoRedondo }, "+")
      )
    );
  }
  function TelaEscala({ onAbrirNotificacoes, onAbrirPerfil, qtdNotificacoes }) {
    const { diasEscala, horasPorPlantao, definirDiaEscala } = useApp();
    const dataHoje = /* __PURE__ */ new Date(HOJE_ISO + "T12:00:00");
    const [mesRef, setMesRef] = useState(() => new Date(dataHoje.getFullYear(),
    dataHoje.getMonth(), 1));
    const [diaSelecionado, setDiaSelecionado] = useState(null);
    const ano = mesRef.getFullYear();
    const mesIndice = mesRef.getMonth();
    const nomeMesBruto = mesRef.toLocaleDateString("pt-BR", { month: "long", year:
    "numeric" });
    const nomeMes = nomeMesBruto.charAt(0).toUpperCase() + nomeMesBruto.slice(1);
    const totalDias = diasNoMes(ano, mesIndice);
    const offsetSemana = primeiroDiaSemanaMes(ano, mesIndice);
    const celulas = [];
    for (let i = 0; i < offsetSemana; i++) celulas.push(null);
    for (let d = 1; d <= totalDias; d++) {
      celulas.push(`${ano}-${String(mesIndice + 1).padStart(2, "0")}-`

```

```

    ${String(d).padStart(2, "0")}`);
  }
  const isosDoMes = celulas.filter(Boolean);
  const plantasNoMes = isosDoMes.filter((iso) => diasEscala[iso] ===
"planta") .length;
  const folgasNoMes = isosDoMes.filter((iso) => diasEscala[iso] === "folga") .length;
  const horasNoMes = plantasNoMes * (horasPorPlanta || 12);
  function buscarProximo(tipo) {
    for (let i = 0; i <= 60; i++) {
      const iso = isoDeHoje(i);
      if (diasEscala[iso] === tipo) {
        if (i === 0) return "Hoje";
        if (i === 1) return "Amanh\u00E3";
        return `${i} dias`;
      }
    }
  }
  return "\u2014";
}
const proximaFolgaTexto = buscarProximo("folga");
let proximoPlantaTexto = "\u2014";
for (let i = 0; i <= 60; i++) {
  const iso = isoDeHoje(i);
  if (diasEscala[iso] === "planta") {
    proximoPlantaTexto = i === 0 ? "Hoje" : i === 1 ? "Amanh\u00E3" :
diaCurto(iso).charAt(0).toUpperCase() + diaCurto(iso).slice(1);
    break;
  }
}
function mudarMes(delta) {
  setMesRef(new Date(ano, mesIndice + delta, 1));
}
const camposModal = [
  {
    chave: "tipo",
    rotulo: "Selecione o tipo",
    tipo: "select",
    opcoes: TIPOS_ESCALA.map((t) => ({ valor: t.valor, rotulo: `${t.emoji}
${t.rotulo}` })))
  }
];
return /* __PURE__ */ React.createElement(
  "div",
  { style: { padding: "0 18px 90px" } },
  /* __PURE__ */ React.createElement(Cabecalho, { titulo: "Escala", subtítulo: "Sua
escala de plant\u00E5es", onAbrirNotificacoes, onAbrirPerfil, qtdNotificacoes }),
  /* __PURE__ */ React.createElement(
    "div",
    { style: { display: "grid", gridTemplateColumns: "1fr 1fr", gap: 10 } },
    /* __PURE__ */ React.createElement(Cartao, { style: { padding: 14 } }, /*
__PURE__ */ React.createElement("p", { style: { fontSize: 11, color: C.textoSuave,
margin: 0 } }, "Pr\u00F3xima folga"), /* __PURE__ */ React.createElement("p", { style:
{ fontSize: 18, fontWeight: 700, color: C.texto, margin: "4px 0 0" } },
proximaFolgaTexto)),
    /* __PURE__ */ React.createElement(Cartao, { style: { padding: 14 } }, /*
__PURE__ */ React.createElement("p", { style: { fontSize: 11, color: C.textoSuave,
margin: 0 } }, "Pr\u00F3ximo plant\u00E3o"), /* __PURE__ */ React.createElement("p", {
style: { fontSize: 18, fontWeight: 700, color: C.texto, margin: "4px 0 0" } },
proximoPlantaTexto)),
    /* __PURE__ */ React.createElement(Cartao, { style: { padding: 14 } }, /*
__PURE__ */ React.createElement("p", { style: { fontSize: 11, color: C.textoSuave,
margin: 0 } }, "Plant\u00E5es no m\u00EAs"), /* __PURE__ */ React.createElement("p", {
style: { fontSize: 18, fontWeight: 700, color: C.texto, margin: "4px 0 0" } },
plantasNoMes)),
    /* __PURE__ */ React.createElement(Cartao, { style: { padding: 14 } }, /*
__PURE__ */ React.createElement("p", { style: { fontSize: 11, color: C.textoSuave,
margin: 0 } }, "Folgas no m\u00EAs"), /* __PURE__ */ React.createElement("p", { style:

```

```

{ fontSize: 18, fontWeight: 700, color: C.texto, margin: "4px 0 0" } }, folgasNoMes))
),
/* @__PURE__ */ React.createElement(Cartao, { style: { padding: 16, marginTop: 10
} }, /* @__PURE__ */ React.createElement("p", { style: { fontSize: 11, color:
C.textoSuave, margin: 0 } }, "Horas trabalhadas no m\xEAs"), /* @__PURE__ */
React.createElement("p", { style: { fontSize: 24, fontWeight: 700, color: C.rosaCha,
margin: "4px 0 0" } }, horasNoMes, "h")),
/* @__PURE__ */ React.createElement(TituloSecao, null, "Calend\xE1rio"),
/* @__PURE__ */ React.createElement(
  Cartao,
  { style: { padding: 16 } },
  /* @__PURE__ */ React.createElement(
    "div",
    { style: { display: "flex", alignItems: "center", justifyContent: "space-
between", marginBottom: 14 } },
    /* @__PURE__ */ React.createElement(BotaoIcone, { icone: ChevronLeft, onClick:
() => mudarMes(-1) }),
    /* @__PURE__ */ React.createElement("h3", { style: { fontSize: 15, fontWeight:
700, color: C.texto, margin: 0 } }, nomeMes),
    /* @__PURE__ */ React.createElement(BotaoIcone, { icone: ChevronRight,
onClick: () => mudarMes(1) })
  ),
  /* @__PURE__ */ React.createElement(
    "div",
    { style: { display: "grid", gridTemplateColumns: "repeat(7, 1fr)", gap: 5 } },
    DIAS_SEMANA_CAL.map((d, i) => /* @__PURE__ */ React.createElement("div", {
key: `cab-${i}`, style: { textAlign: "center", fontSize: 10.5, color: C.textoSuave,
fontWeight: 600, padding: "2px 0 6px" } }, d)),
    celulas.map(
      (iso, i) => iso ? /* @__PURE__ */ React.createElement(
        CelulaDiaEscala,
        {
          key: iso,
          numero: Number(iso.slice(-2)),
          tipoInfo: tipoEscalaPorValor(diasEscala[iso]),
          hoje: iso === HOJE_ISO,
          onClick: () => setDiaSelecionado(iso)
        }
      ) : /* @__PURE__ */ React.createElement("div", { key: `vazio-${i}` })
    )
  ),
  /* @__PURE__ */ React.createElement(
    "div",
    { style: { display: "flex", flexWrap: "wrap", gap: "6px 14px", marginTop: 16 }
},
    TIPOS_ESCALA.map((t) => /* @__PURE__ */ React.createElement("span", { key:
t.valor, style: { display: "flex", alignItems: "center", gap: 5, fontSize: 11, color:
C.textoSuave } }, t.emoji, " ", t.rotulo))
  )
),
diaSelecionado && /* @__PURE__ */ React.createElement(
  ModalFormulario,
  {
    titulo: formatarDataLonga(diaSelecionado),
    campos: camposModal,
    valores: { tipo: diasEscala[diaSelecionado] || "" },
    corDestaque: C.champagne,
    onFechar: () => setDiaSelecionado(null),
    onSalvar: (dados) => {
      definirDiaEscala(diaSelecionado, dados.tipo || null);
      setDiaSelecionado(null);
    },
    onExcluir: diasEscala[diaSelecionado] ? () => {
      definirDiaEscala(diaSelecionado, null);
      setDiaSelecionado(null);
    } : void 0
  }
)

```

```

    }
  )
);
}
function TelaDesafios({ onAbrirNotificacoes, onAbrirPerfil, qtdNotificacoes }) {
  const {
    missoesFeitas,
    alternarMissaoDiaria,
    pontosHoje,
    pontosTotais,
    rodadasCompletas,
    conquistas,
    coposAgua,
    porcoesFrutas,
    ajustarContadorDiario,
    pesoAtual,
    setPesoAtual,
    dieta,
    setRefeicaoDieta,
    historicoDemo
  } = useApp();
  const [subAba, setSubAba] = useState("desafios");
  const [pesoInput, setPesoInput] = useState(String(pesoAtual));
  const qtdMissoesFeitas = MISSOES_DIARIAS.filter((m) => missoesFeitas[m.id]).length;
  const nivel = Math.floor(pontosTotais / 50) + 1;
  const progressoNivel = pontosTotais % 50 / 50 * 100;
  const subs = [
    { id: "desafios", label: "Desafios" },
    { id: "corpo", label: "Corpo" },
    { id: "evolucao", label: "Evolu\xE7\xE3o" }
  ];
  return /* #__PURE__ */ React.createElement(
    "div",
    { style: { padding: "0 18px 90px" } },
    /* #__PURE__ */ React.createElement(Cabecalho, { titulo: "Desafios &
Evolu\xE7\xE3o \u{1F3C6}", subtitulo: "Sua jornada de sa\xFAde e evolu\xE7\xE3o",
onAbrirNotificacoes, onAbrirPerfil, qtdNotificacoes }),
    /* #__PURE__ */ React.createElement(
      Cartao,
      { style: { padding: 18, marginBottom: 14 } },
      /* #__PURE__ */ React.createElement(
        "div",
        { style: { display: "flex", justifyContent: "space-around", textAlign:
"center" } },
        /* #__PURE__ */ React.createElement(
          "div",
          null,
          /* #__PURE__ */ React.createElement("div", { style: { fontSize: 22 } },
            "\u{1F525}"),
          /* #__PURE__ */ React.createElement("p", { style: { fontSize: 20,
fontWeight: 700, color: C.texto, margin: "4px 0 0" } }, pontosHoje),
          /* #__PURE__ */ React.createElement("p", { style: { fontSize: 11, color:
C.textoSuave, margin: 0 } }, "pontos hoje")
        ),
        /* #__PURE__ */ React.createElement(
          "div",
          null,
          /* #__PURE__ */ React.createElement("div", { style: { fontSize: 22 } },
            "\u2705"),
          /* #__PURE__ */ React.createElement("p", { style: { fontSize: 20,
fontWeight: 700, color: C.texto, margin: "4px 0 0" } },
            `${qtdMissoesFeitas}/${MISSOES_DIARIAS.length}`),
          /* #__PURE__ */ React.createElement("p", { style: { fontSize: 11, color:
C.textoSuave, margin: 0 } }, "miss\xF5es")
        ),
        /* #__PURE__ */ React.createElement(

```

```

        "div",
        null,
        /* __PURE__ */ React.createElement("div", { style: { fontSize: 22 } },
        "\u{1F3AF}"),
        /* __PURE__ */ React.createElement("p", { style: { fontSize: 20,
        fontWeight: 700, color: C.texto, margin: "4px 0 0" } }, rodadasCompletas),
        /* __PURE__ */ React.createElement("p", { style: { fontSize: 11, color:
        C.textoSuaVe, margin: 0 } }, "ciclos completos")
    )
  ),
  /* __PURE__ */ React.createElement(
    "div",
    { style: { display: "flex", gap: 6, overflowX: "auto", paddingBottom: 4,
    marginBottom: 14 } },
    subs.map((s) => /* __PURE__ */ React.createElement(
      "button",
      {
        key: s.id,
        onClick: () => setSubAba(s.id),
        style: { whiteSpace: "nowrap", padding: "7px 13px", borderRadius: 999,
        border: "none", fontSize: 11.5, fontWeight: 600, background: subAba === s.id ? C.texto
        : C.rosaClaro, color: subAba === s.id ? "#fff" : C.texto }
      },
      s.label
    ))
  ),
  subAba === "desafios" && /* __PURE__ */ React.createElement(
    React.Fragment,
    null,
    /* __PURE__ */ React.createElement(ContadorDiario, {
      emoji: "\u{1F4A7}",
      titulo: "\xC1gua",
      valor: coposAgua,
      meta: 8,
      unidade: "copos",
      corBarra: C.lilas,
      onSubtrair: () => ajustarContadorDiario("coposAgua", -1, 0, 8),
      onSomar: () => ajustarContadorDiario("coposAgua", 1, 0, 8)
    }),
    /* __PURE__ */ React.createElement(ContadorDiario, {
      emoji: "\u{1F353}",
      titulo: "Frutas",
      valor: porcoesFrutas,
      meta: 3,
      unidade: "por\xE7\xF5es",
      corBarra: C.verde,
      onSubtrair: () => ajustarContadorDiario("porcoesFrutas", -1, 0, 3),
      onSomar: () => ajustarContadorDiario("porcoesFrutas", 1, 0, 3)
    }),
    /* __PURE__ */ React.createElement(
      Cartao,
      { style: { padding: 16, marginBottom: 14 } },
      /* __PURE__ */ React.createElement("p", { style: { fontSize: 14, fontWeight:
      600, color: C.texto, margin: "0 0 12px" } }, "\u{1F37D}\uFE0F Minha Dieta de Hoje"),
      [
        { chave: "cafeDaManha", rotulo: "\u2615 Caf\xE9 da manh\xE3" },
        { chave: "almoco", rotulo: "\u{1F374} Almo\xE7o" },
        { chave: "jantar", rotulo: "\u{1F37D}\uFE0F Jantar" }
      ].map((r, idx) => /* __PURE__ */ React.createElement(
        "div",
        { key: r.chave, style: { marginBottom: idx < 2 ? 12 : 0 } },
        /* __PURE__ */ React.createElement("label", { style: { fontSize: 12,
        fontWeight: 600, color: C.textoSuaVe, display: "block", marginBottom: 5 } },
        r.rotulo),
        /* __PURE__ */ React.createElement("textarea", {

```

```

        value: dieta[r.chave] || "",
        onChange: (e) => setRefeicaoDieta(r.chave, e.target.value),
        placeholder: "O que voc\u00EA vai comer...",
        rows: 2,
        style: { ...estiloInput, resize: "none", fontFamily: "inherit" }
    })
  ))
),
/* @__PURE__ */ React.createElement(
  Cartao,
  { style: { padding: 16, marginBottom: 14 } },
  /* @__PURE__ */ React.createElement(
    "div",
    { style: { display: "flex", justifyContent: "space-between", alignItems:
"center", marginBottom: 10 } },
    /* @__PURE__ */ React.createElement("p", { style: { fontSize: 14,
fontWeight: 600, color: C.texto, margin: 0 } }, `\u{1F525} ${MISSOES_DIARIAS.length}
Miss\u00F5es Di\u00E1rias`),
    /* @__PURE__ */ React.createElement("span", { style: { fontSize: 12, color:
C.textoSuaVe } }, `${qtdMissoesFeitas}/${MISSOES_DIARIAS.length}`)
  ),
  /* @__PURE__ */ React.createElement(ProgressoBarra, { pct: qtdMissoesFeitas /
MISSOES_DIARIAS.length * 100, cor: C.champagne }),
  /* @__PURE__ */ React.createElement(
    "div",
    { style: { marginTop: 14, display: "flex", flexDirection: "column", gap: 10
} },
    MISSOES_DIARIAS.map((m, i) => /* @__PURE__ */ React.createElement(
      "div",
      { key: m.id, style: { display: "flex", alignItems: "center",
justifyContent: "space-between" } },
      /* @__PURE__ */ React.createElement(
        "button",
        {
          onClick: () => alternarMissaoDiaria(m.id),
          style: { display: "flex", alignItems: "center", gap: 12, background:
"none", border: "none", padding: 0, cursor: "pointer", flex: 1, textAlign: "left" }
        },
      /* @__PURE__ */ React.createElement(
        "span",
        {
          style: {
            width: 24,
            height: 24,
            borderRadius: 12,
            flexShrink: 0,
            border: `2px solid ${missoesFeitas[m.id] ? C.champagne :
C.rosaClaro}`,
            background: missoesFeitas[m.id] ? C.champagne : "transparent",
            display: "flex",
            alignItems: "center",
            justifyContent: "center",
            color: "#fff",
            fontSize: 13
          }
        },
        missoesFeitas[m.id] ? "\u2713" : ""
      ),
      /* @__PURE__ */ React.createElement("span", { style: { fontSize: 13.5,
color: C.texto } }, `${i + 1}. ${m.texto}`)
    )
  ))
  ),
  /* @__PURE__ */ React.createElement(
    Cartao,

```

```

    { style: { padding: 18 } },
    /* @__PURE__ */ React.createElement(
      "div",
      { style: { display: "flex", alignItems: "center", gap: 6, marginBottom: 12 }
    },
      /* @__PURE__ */ React.createElement("span", { style: { fontSize: 15 } },
        "\u{1F396}\uFE0F"),
        /* @__PURE__ */ React.createElement("span", { style: { fontSize: 12.5,
fontWeight: 700, color: C.textoSuave, letterSpacing: 0.5 } }, "SISTEMA DE
PONTUA\uxC7\uC30"),
      ),
      /* @__PURE__ */ React.createElement("p", { style: { textAlign: "center",
fontSize: 30, fontWeight: 700, color: C.lilas, margin: 0 } }, pontosTotais),
      /* @__PURE__ */ React.createElement("p", { style: { textAlign: "center",
fontSize: 11.5, color: C.textoSuave, margin: "0 0 16px" } }, "pontos totais"),
      /* @__PURE__ */ React.createElement(
        "div",
        { style: { display: "flex", justifyContent: "space-around", textAlign:
"center", marginBottom: 14 } },
        /* @__PURE__ */ React.createElement(
          "div",
          null,
          /* @__PURE__ */ React.createElement("div", { style: { fontSize: 18 } },
            "\u2B50"),
            /* @__PURE__ */ React.createElement("p", { style: { fontSize: 14,
fontWeight: 700, color: C.texto, margin: "4px 0 0" } }, `N\uxEDvel ${nivel}`)
          ),
          /* @__PURE__ */ React.createElement(
            "div",
            null,
            /* @__PURE__ */ React.createElement("div", { style: { fontSize: 18 } },
              "\u26A1"),
              /* @__PURE__ */ React.createElement("p", { style: { fontSize: 14,
fontWeight: 700, color: C.texto, margin: "4px 0 0" } }, pontosHoje),
              /* @__PURE__ */ React.createElement("p", { style: { fontSize: 10.5, color:
C.textoSuave, margin: 0 } }, "pontos hoje")
            ),
            /* @__PURE__ */ React.createElement(
              "div",
              null,
              /* @__PURE__ */ React.createElement("div", { style: { fontSize: 18 } },
                "\u{1F3AF}"),
                /* @__PURE__ */ React.createElement("p", { style: { fontSize: 14,
fontWeight: 700, color: C.texto, margin: "4px 0 0" } }, rodadasCompletas),
                /* @__PURE__ */ React.createElement("p", { style: { fontSize: 10.5, color:
C.textoSuave, margin: 0 } }, "ciclos")
              )
            ),
            /* @__PURE__ */ React.createElement("div", { style: { padding: "0 20px",
marginBottom: 16 } }, /* @__PURE__ */ React.createElement(ProgressoBarra, { pct:
progressoNivel, cor: C.champagne })),
            /* @__PURE__ */ React.createElement("p", { style: { fontSize: 12, color:
C.textoSuave, margin: "0 0 8px" } }, "Conquistas desbloqueadas"),
            conquistas.length === 0 ? /* @__PURE__ */ React.createElement("p", { style: {
fontSize: 12, color: C.textoSuave, fontStyle: "italic", margin: 0 } }, "Complete sua
primeira miss\uxE3o para desbloquear!") : /* @__PURE__ */ React.createElement(
              "div",
              { style: { display: "flex", flexWrap: "wrap", gap: 8 } },
              conquistas.includes("primeira_missao") && /* @__PURE__ */
React.createElement(Pill, { cor: C.champagne }, "\u{1F3AF} Primeira Miss\uxE3o")
            )
          ),
        ),
      ),
      subAba === "corpo" && /* @__PURE__ */ React.createElement(
        Cartao,
        { style: { padding: 18 } },

```

```

        /* __PURE__ */ React.createElement("p", { style: { fontSize: 12, color:
C.textoSuaVe, margin: 0 } }, "Peso atual"),
        /* __PURE__ */ React.createElement("p", { style: { fontSize: 30, fontWeight:
700, color: C.texto, margin: "4px 0 16px" } }, `${pesoAtual}kg`),
        /* __PURE__ */ React.createElement(
            "div",
            { style: { display: "flex", gap: 8 } },
            /* __PURE__ */ React.createElement("input", {
                type: "number",
                style: estiloInput,
                value: pesoInput,
                onChange: (e) => setPesoInput(e.target.value)
            }),
            /* __PURE__ */ React.createElement(
                "button",
                {
                    onClick: () => {
                        const v = Number(pesoInput);
                        if (!Number.isNaN(v) && v > 0) setPesoAtual(v);
                    },
                    style: { padding: "0 18px", borderRadius: 14, border: "none", background:
C.champagne, color: "#fff", fontSize: 13, fontWeight: 600 }
                },
                "Salvar"
            )
        ),
        subAba === "evolucao" && /* __PURE__ */ React.createElement(
            Cartao,
            { style: { padding: 18 } },
            /* __PURE__ */ React.createElement("p", { style: { fontSize: 13, fontWeight:
600, color: C.texto, margin: "0 0 12px" } }, "Evolu\u00E7\u00E3o dos \xFAltimos dias"),
            /* __PURE__ */ React.createElement(
                "div",
                { style: { width: "100%", height: 180 } },
                /* __PURE__ */ React.createElement(
                    ResponsiveContainer,
                    { width: "100%", height: "100%" },
                    /* __PURE__ */ React.createElement(
                        LineChart,
                        { data: historicoDemo },
                        /* __PURE__ */ React.createElement(CartesianGrid, { stroke: C.rosaClaro,
vertical: false }),
                        /* __PURE__ */ React.createElement(XAxis, { dataKey: "dia", tick: {
fontSize: 10, fill: C.textoSuaVe } }),
                        /* __PURE__ */ React.createElement(YAxis, { tick: { fontSize: 10, fill:
C.textoSuaVe }, width: 28 }),
                        /* __PURE__ */ React.createElement(Tooltip, null),
                        /* __PURE__ */ React.createElement(Line, { type: "monotone", dataKey:
"pct", stroke: C.lilas, strokeWidth: 2, dot: { r: 3 } })
                    )
                )
            )
        ),
    );
}
function TelaPerfil({ onFechar }) {
    const { perfil, editarPerfil, tarefas, habitos, metas } = useApp();
    const [editando, setEditando] = useState(false);
    const campos = [
        { chave: "nome", rotulo: "Nome", tipo: "texto" },
        { chave: "horarioInicio", rotulo: "Hor\u00E1rio de in\u00E9dio do dia", tipo: "hora"
    },
        { chave: "objetivos", rotulo: "Objetivo principal", tipo: "texto" }
    ];
    const lembretes = [

```

```

        ..tarefas.filter((t) => t.lembrete).map((t) => ({ tipo: "Tarefa", nome: t.titulo,
quando: t.hora || t.data })),
        ..habitros.filter((h) => h.lembrete).map((h) => ({ tipo: "H\u00c1bito", nome:
h.nome, quando: h.frequencia })))
    ];
    return /* __PURE__ */ React.createElement("div", { style: { position: "fixed",
inset: 0, background: C.bg, zIndex: 60, overflowY: "auto" } }, /* __PURE__ */
React.createElement("div", { style: { maxWidth: 384, margin: "0 auto", padding: "22px
20px 40px" } }, /* __PURE__ */ React.createElement("button", { onClick: onFechar,
style: { background: "none", border: "none", color: C.textoSuaVe, fontSize: 13,
marginBottom: 12 } }, "\u2190 Voltar"), /* __PURE__ */ React.createElement("div", {
style: { textAlign: "center", marginBottom: 20 } }, /* __PURE__ */
React.createElement("div", { style: { width: 72, height: 72, borderRadius: 36,
background: `linear-gradient(135deg, ${C.rosaCh\u00e1}, ${C.lilas})`, display: "flex",
alignItems: "center", justifyContent: "center", fontSize: 32, margin: "0 auto 10px" }
}, perfil?.avatar), /* __PURE__ */ React.createElement("h2", { style: { fontSize: 18,
fontWeight: 700, color: C.texto, margin: 0 } }, perfil?.nome), /* __PURE__ */
React.createElement("p", { style: { fontSize: 12, color: C.textoSuaVe, margin: "4px 0
0" } }, perfil?.objetivos)), /* __PURE__ */ React.createElement(Cartao, { style: {
padding: 16, marginBottom: 16 } }, /* __PURE__ */ React.createElement("div", { style:
{ display: "flex", justifyContent: "space-between", marginBottom: 10 } }, /* __PURE__
*/ React.createElement("p", { style: { fontSize: 13, fontWeight: 600, color: C.texto,
margin: 0 } }, "Prefer\u00eancia"), /* __PURE__ */ React.createElement("button", {
onClick: () => setEditando(true), style: { background: "none", border: "none", color:
C.champagne, fontSize: 12, fontWeight: 600 } }, "Editar")), /* __PURE__ */
React.createElement("p", { style: { fontSize: 12, color: C.textoSuaVe, margin: "4px 0"
} }, "In\u00e9dio do dia: ", perfil?.horarioInicio)), /* __PURE__ */
React.createElement(TituloSecao, null, "Notifica\u00e7\u00f5es e lembretes"),
lembretes.length === 0 ? /* __PURE__ */ React.createElement(EstadoVazio, { texto:
"Nenhum lembrete configurado." }) : /* __PURE__ */ React.createElement("div", {
style: { display: "flex", flexDirection: "column", gap: 8 } }, lembretes.map((l, i) =>
/* __PURE__ */ React.createElement(ItemLinha, { key: i, icone: Bell, corIcone:
C.champagne, titulo: l.nome, subtitulo: `${l.tipo} \u00b7 ${l.quando}` }))), /*
__PURE__ */ React.createElement(TituloSecao, null, "Configura\u00e7\u00f5es"), /*
__PURE__ */ React.createElement("div", { style: { display: "flex", flexDirection:
"column", gap: 8 } }, /* __PURE__ */ React.createElement(ItemLinha, { icone: User,
corIcone: C.malva, titulo: "Editar perfil", onClick: () => setEditando(true), direita:
/* __PURE__ */ React.createElement(ChevronRight, { size: 16, color: C.textoSuaVe })
}), /* __PURE__ */ React.createElement(ItemLinha, { icone: Settings, corIcone:
C.textoSuaVe, titulo: "Categorias e rotinas", subtitulo: "Personalize suas
categorias", direita: /* __PURE__ */ React.createElement(ChevronRight, { size: 16,
color: C.textoSuaVe }) }))), editando && /* __PURE__ */ React.createElement(
    ModalFormulario,
    {
        titulo: "Editar perfil",
        campos,
        valores: perfil,
        corDestaque: C.rosaCh\u00e1,
        onFechar: () => setEditando(false),
        onSalvar: (d) => {
            editarPerfil(d);
            setEditando(false);
        }
    }
));
}
function TelaNotificacoes({ onFechar }) {
    const { tarefas, habitros } = useApp();
    const itens = [
        ..tarefas.filter((t) => t.lembrete && !tarefaConcluidaEm(t, HOJE_ISO)).map((t) =>
({ nome: t.titulo, quando: `Hoje \u00c0s ${t.hora || "\u002014"}`, tipo: "Tarefa" })),
        ..habitros.filter((h) => h.lembrete && !h.historico[HOJE_ISO]).map((h) => ({ nome:
h.nome, quando: h.frequencia, tipo: "H\u00c1bito" })))
    ];
    return /* __PURE__ */ React.createElement("div", { style: { position: "fixed",
inset: 0, background: "rgba(91,75,87,0.35)", zIndex: 60, display: "flex",

```

```

justifyContent: "center", alignItems: "flex-end" }, onClick: onFechar }, /* @__PURE__
*/ React.createElement("div", { style: { width: "100%", maxWidth: 384, background:
"#fff", borderRadius: "28px 28px 0 0", padding: 22, maxHeight: "75vh", overflowY:
"auto" }, onClick: (e) => e.stopPropagation() }, /* @__PURE__ */
React.createElement("div", { style: { display: "flex", justifyContent: "space-
between", marginBottom: 14 } }, /* @__PURE__ */ React.createElement("h3", { style: {
fontSize: 16, fontWeight: 600, color: C.texto, margin: 0 } }, "Notifica\uE7\uF5es"),
/* @__PURE__ */ React.createElement("button", { onClick: onFechar, style: {
background: "none", border: "none" } }, /* @__PURE__ */ React.createElement(X, { size:
20, color: C.textoSuaVe }), itens.length === 0 ? /* @__PURE__ */
React.createElement(EstadoVazio, { texto: "Voc\uEA est\uE1 em dia com tudo!\u{1F389}"
}) : /* @__PURE__ */ React.createElement("div", { style: { display: "flex",
flexDirection: "column", gap: 8 } }, itens.map((it, i) => /* @__PURE__ */
React.createElement(ItemLinha, { key: i, icone: Bell, corIcone: C.champagne, titulo:
it.nome, subtítulo: `${it.tipo} \xB7 ${it.quando}` })))));
}
function AppProvider({ children }) {
  const [estado, setEstado] = useState(() => {
    const salvo = carregarEstadoSalvo();
    return salvo ? { ...dadosIniciais(), ...salvo } : dadosIniciais();
  });
  useEffect(() => {
    salvarEstado(estado);
  }, [estado]);
  const acoes = {
    editarPerfil: (novo) => setEstado((e) => ({ ...e, perfil: { ...e.perfil, ...novo }
  }))),
    setHumorHoje: (v) => setEstado((e) => ({ ...e, humorHoje: v })),
    adicionarTarefa: (t) => setEstado((e) => ({ ...e, tarefas: [...e.tarefas, { id:
uid(), historico: {}, concluida: false, ...t } }))),
    editarTarefa: (id, novo) => setEstado((e) => ({ ...e, tarefas: e.tarefas.map((t)
=> t.id === id ? { ...t, ...novo } : t }))),
    removerTarefa: (id) => setEstado((e) => ({ ...e, tarefas: e.tarefas.filter((t) =>
t.id !== id }))),
    alternarTarefa: (id, iso) => setEstado((e) => ({
      ...e,
      tarefas: e.tarefas.map((t) => {
        if (t.id !== id) return t;
        if (t.recorrente) return { ...t, historico: { ...t.historico, [iso]:
!t.historico[iso] } };
        return { ...t, concluida: !t.concluida };
      })
    }))),
    adicionarHabitto: (h) => setEstado((e) => ({ ...e, habitos: [...e.habitos, { id:
uid(), historico: {}, ...h } }))),
    editarHabitto: (id, novo) => setEstado((e) => ({ ...e, habitos: e.habitos.map((h)
=> h.id === id ? { ...h, ...novo } : h }))),
    removerHabitto: (id) => setEstado((e) => ({ ...e, habitos: e.habitos.filter((h) =>
h.id !== id }))),
    alternarHabitto: (id, iso) => setEstado((e) => ({ ...e, habitos: e.habitos.map((h)
=> h.id === id ? { ...h, historico: { ...h.historico, [iso]: !h.historico[iso] } } :
h }))),
    adicionarAutocuidado: (a) => setEstado((e) => ({ ...e, autocuidado:
[...e.autocuidado, { id: uid(), feitoHoje: false, ...a } ] }))),
    alternarAutocuidado: (id) => setEstado((e) => ({ ...e, autocuidado:
e.autocuidado.map((a) => a.id === id ? { ...a, feitoHoje: !a.feitoHoje } : a }))),
    removerAutocuidado: (id) => setEstado((e) => ({ ...e, autocuidado:
e.autocuidado.filter((a) => a.id !== id }))),
    adicionarDiario: (d) => setEstado((e) => ({ ...e, diario: [...e.diario, { id:
uid(), ...d } ] }))),
    removerDiario: (id) => setEstado((e) => ({ ...e, diario: e.diario.filter((d) =>
d.id !== id }))),
    adicionarDesejo: (d) => setEstado((e) => ({ ...e, desejos: [...e.desejos, { id:
uid(), ...d } ] }))),
    removerDesejo: (id) => setEstado((e) => ({ ...e, desejos: e.desejos.filter((d) =>
d.id !== id }))),

```

```

    adicionarProjeto: (p) => setEstado((e) => ({ ...e, projetos: [...e.projetos, { id:
uid(), ...p }] })),
    editarProjeto: (id, novo) => setEstado((e) => ({ ...e, projetos:
e.projetos.map((p) => p.id === id ? { ...p, ...novo } : p) })),
    removerProjeto: (id) => setEstado((e) => ({ ...e, projetos: e.projetos.filter((p)
=> p.id !== id) })),
    adicionarMeta: (m) => setEstado((e) => ({ ...e, metas: [...e.metas, { id: uid(),
...m }] })),
    editarMeta: (id, novo) => setEstado((e) => ({ ...e, metas: e.metas.map((m) => m.id
=== id ? { ...m, ...novo } : m) })),
    removerMeta: (id) => setEstado((e) => ({ ...e, metas: e.metas.filter((m) => m.id
!== id) })),
    adicionarTransacao: (t) => setEstado((e) => ({ ...e, transacoes: [...e.transacoes,
{ id: uid(), ...t }] })),
    removerTransacao: (id) => setEstado((e) => ({ ...e, transacoes:
e.transacoes.filter((t) => t.id !== id) })),
    setOrcamentoMensal: (v) => setEstado((e) => ({ ...e, orcamentoMensal: v })),
    adicionarMetaFinanceira: (m) => setEstado((e) => ({ ...e, metasFinanceiras:
[...e.metasFinanceiras, { id: uid(), ...m }] })),
    contribuirMetaFinanceira: (id, valor) => setEstado((e) => ({ ...e,
metasFinanceiras: e.metasFinanceiras.map((m) => m.id === id ? { ...m, valorAtual:
m.valorAtual + valor } : m) })),
    removerMetaFinanceira: (id) => setEstado((e) => ({ ...e, metasFinanceiras:
e.metasFinanceiras.filter((m) => m.id !== id) })),
    definirDiaEscala: (iso, tipo) => setEstado((e) => {
    const diasEscala = { ...e.diasEscala };
    if (tipo === null || tipo === void 0 || tipo === "") {
      delete diasEscala[iso];
    } else {
      diasEscala[iso] = tipo;
    }
    return { ...e, diasEscala };
  })),
    setHorasPorPlantao: (horas) => setEstado((e) => ({ ...e, horasPorPlantao: horas
})),
    alternarMissaoDiaria: (id) => setEstado((e) => {
    const feitas = { ...e.missoesFeitas };
    const estavaFeita = !!feitas[id];
    if (estavaFeita) {
      delete feitas[id];
    } else {
      feitas[id] = true;
    }
    const delta = estavaFeita ? -1 : 1;
    let conquistas = e.conquistas;
    if (!estavaFeita && !conquistas.includes("primeira_missao")) {
      conquistas = [...conquistas, "primeira_missao"];
    }
    const completouTudo = !estavaFeita && MISSOES_DIARIAS.every((m) =>
feitas[m.id]);
    if (completouTudo) {
      return {
        ...e,
        missoesFeitas: {},
        pontosHoje: e.pontosHoje + delta,
        pontosTotais: e.pontosTotais + delta,
        rodadasCompletas: e.rodadasCompletas + 1,
        conquistas
      };
    }
    return {
      ...e,
      missoesFeitas: feitas,
      pontosHoje: Math.max(0, e.pontosHoje + delta),
      pontosTotais: Math.max(0, e.pontosTotais + delta),
      conquistas
    };
  }));

```

```

    });
  },
  ajustarContadorDiario: (chave, delta, min, max) => setEstado((e) => ({ ...e,
[chave]: Math.min(max, Math.max(min, (e[chave] || 0) + delta)) })),
  setPesoAtual: (valor) => setEstado((e) => ({ ...e, pesoAtual: valor })),
  setRefeicaoDieta: (chave, valor) => setEstado((e) => ({ ...e, dieta: { ...e.dieta,
[chave]: valor } })),
  });
  return /* __PURE__ */ React.createElement(AppCtx.Provider, { value: { ...estado,
...acoes } }, children);
}
function Conteudo() {
  const { perfil, editarPerfil, tarefas, habitos } = useApp();
  const [aba, setAba] = useState("inicio");
  const [mostrarPerfil, setMostrarPerfil] = useState(false);
  const [mostrarNotif, setMostrarNotif] = useState(false);
  if (!perfil) {
    return /* __PURE__ */ React.createElement(Onboarding, { onConcluir: (dados) =>
editarPerfil(dados) });
  }
  const qtdNotif = tarefas.filter((t) => t.lembrete && !tarefaConcluidaEm(t,
HOJE_ISO)).length + habitos.filter((h) => h.lembrete &&
!h.historico[HOJE_ISO]).length;
  return /* __PURE__ */ React.createElement("div", { style: { minHeight: "100vh",
background: C.bg, fontFamily: "'Poppins', 'Segoe UI', sans-serif" } }, /* __PURE__ */
React.createElement("div", { style: { maxWidth: 384, margin: "0 auto", position:
"relative" } }, aba === "inicio" && /* __PURE__ */ React.createElement(
  TelaInicio,
  {
    irPara: setAba,
    onAbrirNotificacoes: () => setMostrarNotif(true),
    onAbrirPerfil: () => setMostrarPerfil(true),
    qtdNotificacoes: qtdNotif
  }
), aba === "dia" && /* __PURE__ */ React.createElement(TelaMeuDia, {
onAbrirNotificacoes: () => setMostrarNotif(true), onAbrirPerfil: () =>
setMostrarPerfil(true), qtdNotificacoes: qtdNotif }), aba === "vida" && /* __PURE__
*/ React.createElement(TelaVida, { onAbrirNotificacoes: () => setMostrarNotif(true),
onAbrirPerfil: () => setMostrarPerfil(true), qtdNotificacoes: qtdNotif }), aba ===
"dinheiro" && /* __PURE__ */ React.createElement(TelaDinheiro, { onAbrirNotificacoes:
() => setMostrarNotif(true), onAbrirPerfil: () => setMostrarPerfil(true),
qtdNotificacoes: qtdNotif }), aba === "escala" && /* __PURE__ */
React.createElement(TelaEscala, { onAbrirNotificacoes: () => setMostrarNotif(true),
onAbrirPerfil: () => setMostrarPerfil(true), qtdNotificacoes: qtdNotif }), aba ===
"desafios" && /* __PURE__ */ React.createElement(TelaDesafios, { onAbrirNotificacoes:
() => setMostrarNotif(true), onAbrirPerfil: () => setMostrarPerfil(true),
qtdNotificacoes: qtdNotif }), /* __PURE__ */ React.createElement(NavegacaoInferior, {
ativa: aba, onMudar: setAba }), mostrarPerfil && /* __PURE__ */
React.createElement(TelaPerfil, { onFechar: () => setMostrarPerfil(false) }),
mostrarNotif && /* __PURE__ */ React.createElement(TelaNotificacoes, { onFechar: ()
=> setMostrarNotif(false) })))));
}
function MeuPlanner() {
  return /* __PURE__ */ React.createElement(AppProvider, null, /* __PURE__ */
React.createElement(Conteudo, null));
}
const elementoRaiz = document.getElementById("root");
const raiz = createRoot(elementoRaiz);
raiz.render(/* __PURE__ */ React.createElement(MeuPlanner, null));

```